Content-Aware Diffing: Edit Scripts, Visualisations, and a Living Ground Truth

Marko Ivanković
Universität Passau
Passau, Germany
marko@ivankovic.me

Goran Petrović
Universität Passau
Passau, Germany
stgpetrovic@gmail.com

René Just
University of Washington
Seattle, USA
rjust@cs.washington.edu

Gordon Fraser
Universität Passau
Passau, Germany
gordon.fraser@uni-passau.de

Abstract

Diffing, the process of comparing two versions of a file and showing what changed between them, underlies version control and code review. The files that commits change hold content of many types: text, code, pictures, archives, documents, fonts. For each type a diffing tool does two things that are usually treated as one. It computes a *delta*, an edit script by which a machine converts one version into the other, which is judged by its correctness and its cost. It also produces a *visualisation*, an account that a human reads in order to understand what happened, which can be judged only against human reading. We study the distance between the two on real changes to all content, with code as the content type that has the deepest ground truth.

We built a living ground-truth dataset, the test corpus of an open-source project with 1,313 GitHub stars, to which ground truth was added in each of the 19 weeks since its first hand mapping. It holds 1,699 real code changes in 25 languages with a human-authored node mapping, 1,439 of them with human-painted renderings, and 194 changes to pictures, cursors, fonts, archives, message catalogs and PDFs judged by hand, together with 31 invariants that a visualisation has to obey and a delta does not. Git judges 9.1% of the 2,021,166 files that recent commits of 7,444 repositories change to be binary, and shows a developer only that they differ. Painted as it stands, the line edit script of `diff` breaks at least one invariant on 88.5% of changes, and the human-preferred tree mapping, painted node by node, still does on 73.5%. On 13.4% of code changes the preferred correspondence cannot be expressed by the common operations (insert, delete, update and move); 32.4% of syntax-tree nodes never reach the screen; and 30.9% of the judged changes to other content, every one a non-empty byte delta, show a reader nothing that anyone chose to change. A visualisation is also not unique: 62.5% of painted changes have a sparse and a generous correct rendering, the generous one painting 1.34 times the bytes. A whole-tree tree-edit-distance computation completes within one second for only 46.8% of 19,798 sampled code file pairs, and no configuration of eight established tools renders more than 79% of code changes perfectly.

As a tool contribution, we present OmniDiff, which diffs a file by what it holds: text, code in 26 languages, and six families of other content. It diffs 98.8% of the changed files as text or by their content. On code, it maps every visible node as the human did on 84.0% of changes, and on 194 judged changes to other content its verdict of how much changed matches a human's on 75%.

CCS Concepts

• **Software and its engineering → Software maintenance tools**; *Empirical software validation.*

Keywords

content-aware diffing, edit scripts, diff visualisation, source code differencing, binary file differencing, ground-truth dataset, empirical software engineering

ACM Reference Format:
Marko Ivanković, Goran Petrović, René Just, and Gordon Fraser. 2026. Content-Aware Diffing: Edit Scripts, Visualisations, and a Living Ground Truth. In *Proceedings of Make sure to enter the correct conference title from your rights confirmation email (Conference'26).* ACM, New York, NY, USA, 21 pages. https://doi.org/XXXXXXX.XXXXXXX

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
Conference'26, June 03–05, 2026, Ludbreg, Croatia
© 2026 Copyright held by the owner/author(s). Publication rights licensed to ACM.
ACM ISBN 978-1-4503-XXXX-X/26/06
https://doi.org/XXXXXXX.XXXXXXX

1 Introduction

Diffing, the process of comparing two versions of a file and showing what changed between them, underlies modern version control and, especially, modern code review. During a review, developers read the diff to see what changed and what stayed the same, so the quality of the diff directly shapes the ease and quality of the review. Whether that reading succeeds depends on how the change is shown, not only on the change. Reviewers name understanding the change as the main challenge of code review [6], and ask for tools that show a change's structure rather than its text delta [56]. Controlled experiments show the presentation alone affecting the outcome. Files shown later in a review receive fewer comments, and a seeded defect in the last file has 64% lower odds of being found than the same defect in the first [26, 52]. The unified view of a diff costs less visual effort and finds more bugs than the split view of the same diff [2]. A change decomposed into its separate

DRAFT

Before	After

`def save(rec):`
`  db.begin()`
`  db.put(rec)`
`  db.commit()`
	`def save(rec):`
	`  if rec.dirty:`
	`  db.begin()`
	`  db.put(rec)`
	`  db.commit()`

Figure 1: Wrapping a block in a condition. Only whitespace (indentation) changed, but a line-based diff reports all of it as deleted and reinserted. An AST diff can report one insertion and a move.

concerns is reviewed with fewer wrongly reported issues than the same change tangled [7, 17, 30]. A reviewer's working-memory capacity predicts whether they find a *delocalised* defect, one whose parts the diff shows far apart [8, 65].

The cost of a diff that is hard to read is paid in time and in quality. At Google, useful comments fall and review latency rises with the size of a change [54], and Mozilla's contributors name size-related factors as the largest driver of review latency [37]. Components with low review participation ship more post-release defects [45]. Practitioners rate the reaction time of review tooling as very important [40], and at Meta, developer satisfaction with code review falls with the time the slowest quarter of a developer's changes wait for review [11]. Where a diff shows a refactoring as a refactoring rather than as deleted and inserted lines, reviewers report lower cognitive effort and read fewer lines [12, 28], and syntactic deltas modelled on how developers edit are judged more understandable than the cheapest tree edit [16].

What a commit changes is content of many types. In the corpus of Section 3, the files that recent commits change are text of every kind (code, data, configuration and documentation), and they are also pictures, which change in 725 of 7,352 repositories, archives, PDFs, fonts, compiled translation catalogs and cursor themes. The diff that almost all developers and platforms use today treats every one of them the same way. A file is a sequence of text lines, and the tool computes an edit script between the two sequences. A file whose bytes git judges binary, chiefly by a NUL byte, is reported as `Binary files differ` and nothing more [57]; that is 9.1% of the files the commits change, and a reviewer of such a change learns that it happened and not what it was. For text the line edit script is cheap to compute and correct, and it is still a poor account of many changes. A whole class of common code changes—extracting code into a function, wrapping a block in a condition, reordering declarations—is a move of existing code, which a line script can state only as a deletion plus an insertion. Figure 1 shows a small example: wrapping three unchanged lines in a conditional, which a line-based tool reports as three lines deleted and four inserted, and which a reader has to diff again in their head to see that only the wrapper is new.

We argue that these are two faces of one problem, and that it is not specific to code. An edit script, which we call a *delta*, is a recipe by which a machine converts one version of a file into the other. It is judged by whether it is correct and by how little it costs, and a line script, a tree edit script and a binary delta are all good at exactly that. A *visualisation* is what a human needs in order to understand what happened, and it is judged against human reading. The two come apart for every content type. The cheapest tree edit script pairs two unrelated `return` keywords; the smallest byte delta between two PNG files says nothing a viewer can use; a picture re-encoded without loss has a large delta and no visible change. A visualisation may even be an account that is not minimal and cannot be executed: a move shown as a move, a rename shown word by word, whitespace suppressed, a picture's change described by how much of it a viewer would see. Section 2 develops the distinction, and the rest of the paper measures it.

Measuring it needs a ground truth that records what humans consider the right account of a change, for more than one content type, and that can be kept honest as it grows. We built one as the test corpus of an open-source diffing tool that is under active development and in use, so the dataset is not a snapshot: it grows by sampling, by hand curation and from the issues users report, and 31 machine-checked invariants hold its hand-made answers to one convention.

We examined five groups of research questions. Each group is answered in its own section, in the order listed here. Wherever a question can be asked of all content we ask it of all content and report code as one content type among the others; where our data exists only for code, which has the deepest ground truth, the question says so.

- **Research Question Group 1 (Content):** What content do real-world changes touch? How large are real-world changes?
- **Research Question Group 2 (Delta versus visualisation):** How often does a delta, painted as it stands, break the rules that a visualisation obeys? How much of a delta reaches the reader? What percentage of real-world code changes have a human-preferred node mapping that the common edit operations (Insert, Delete, Update, Move) cannot express? What percentage have a human-preferred mapping whose cost is higher than that of the cheapest mapping found?
- **Research Question Group 3 (Uniqueness):** What percentage of real-world code changes have multiple optimal node mappings? What percentage have multiple equally correct visualisations, and how do those differ?
- **Research Question Group 4 (Exactness):** What percentage of real-world source-code changes can a single, whole-tree tree-edit-distance computation complete within one second?[1]
- **Research Question Group 5 (State of the art):** What do established tools show for each type of content? How accurately do current state-of-the-art code diffing tools find the human-preferred visualisation? How fast are they?

This paper makes the following contributions:

- **Theory:** It separates the delta from the visualisation of a change, defines delta, mapping, painting and the projection from a mapping to a painting for content in general, and states which properties each has to satisfy (Section 2).

[1] One second is where Nielsen's response-time limits put the boundary between an interaction that keeps a user's train of thought and one that visibly interrupts it [47].

PREPRINT

- **Dataset:** It presents a living ground-truth dataset: 1,699 real-world code changes in 25 languages, each with a hand-made node mapping, 1,439 of them with hand-painted renderings in up to two presets, and 194 changes to pictures, cursors, fonts, archives, compiled message catalogs and PDFs, each judged by hand for how much it changed. The changes were drawn from 7,444 open-source repositories by three sampling methods, and a fourth part is the Defects4J [35] changes of the node-mapping oracle of Alikhanifard and Tsantalis [3], solved again by us for comparison with prior work. The dataset, the tool used to create it and its entire history are freely available (Section 3).
- **Invariants:** It states 31 invariants that every ground-truth answer obeys, checked by machine. They are rules of visualisation that no delta needs, so the number of them an account of a change breaks measures its distance from a visualisation (Sections 3.5 and 5).
- **Evaluation:** It reports a census of all 2,018,010 files the corpus's recent commits change, by what each file holds, an analysis of the shape of real-world changes that is independent of any implementation, and a quantified comparison of eight established diffing tools against the human ground truth (Sections 4–8).
- **Tool:** It presents OmniDiff, a tool that diffs text, code and six families of other content, each by what it holds, built from the results above (Section 9).

2 Background: Deltas and Visualisations

A diffing tool produces two artifacts that the literature and most tools treat as one: a delta, which a machine executes, and a visualisation, which a human reads. This section reviews the work on each, defines both for content in general, and lists the tools the paper measures.

2.1 Deltas

Sequences. The string-to-string correction problem of Wagner and Fischer [63] asks for the cheapest sequence of character insertions, deletions and changes that turns one string into another, and solves it by dynamic programming. Unix `diff` applies the same idea to lines: Hunt and McIlroy [34] compute a longest common subsequence of the two files' lines and report the rest as deleted and inserted. Myers' $O(ND)$ algorithm [46] finds a shortest such script in time that scales with the size of the script rather than of the inputs, and underlies most line-based tools in use today.

Block moves. A shortest insert-and-delete script is not the shortest recipe once copying is allowed. Tichy [59] extends string correction with block moves and shows that the resulting scripts are shorter than those based on a longest common subsequence, and Heckel [29] anchors a comparison on the lines that occur exactly once in both files, which detects blocks that moved.

Trees. Tai [55] generalises string correction to ordered labelled trees, with node relabelling, deletion and insertion as the operations. Zhang and Shasha [68] give the classic dynamic-programming algorithm for this tree edit distance; it is exact, but its keyroot decomposition repeats work across overlapping subproblems. RTED [49] and its successor APTED [50] choose a decomposition strategy per subproblem, which keeps the $O(n^3)$ worst case and lowers the runtime on real trees markedly; Bille [9] surveys the field. Chawathe et al. [15] split the problem in two, a split this paper builds on: first find a good *matching* between the nodes of the two trees, then derive a minimum-cost edit script of insert, delete, update and *move* operations that conforms to it. Source-code differencing follows that split. ChangeDistiller [25] adapts the matching to abstract syntax trees (ASTs), and GumTree [23] matches large isomorphic subtrees top-down and containers bottom-up. Later work shortens the scripts further by finding more moves [18], by matching names and partial subtrees [27], by letting unchanged lines constrain the node matching [44], or by replacing GumTree's costly recovery phase with a simpler heuristic [21]. Most of this work measures quality by the length of the edit script; Fan et al. [24] compare three of the algorithms differentially and find that each produces inaccurate mappings for a sizeable share of file revisions.

Bytes. Delta compression drops the reader altogether. The rsync algorithm [60] sends only the parts of a file the receiver lacks, VCDIFF [38] is a portable format of copy and add instructions, bsdiff [51] finds approximately matching regions of two executables by suffix sorting, and git stores the objects of a packfile as chains of such deltas [58]. Hunt et al. [33] compare delta algorithms on real releases by the two criteria that matter for a delta, size and speed, and report that a byte-level algorithm produces smaller deltas than line-based `diff`.

Other content. Content that is not a sequence of characters has deltas of its own kind. The delta of two pictures is the set of pixels that differ, the delta of two archives is the list of members added, removed and changed, and the delta of two fonts is a list of tables or glyphs. Each is as mechanical as a line script, and each is correct by construction.

2.2 Visualisations

Text and code. That the display of a difference is a problem of its own is as old as syntactic differencing: Yang's program comparison tool [66] matches two parse trees and then pretty-prints both programs side by side with the differing parts highlighted. Later work moves the account of a change further from the edit script and closer to the reader. Ldiff [14] and LHDiff [5] tell changed lines from added and deleted ones and follow lines that moved. srcDiff [16] decides by rules modelled on how developers edit whether two differing elements are one construct changed or one construct replaced by another, and reports deltas that developers judge more understandable than the cheapest ones. ClDiff [31] groups fine-grained differences into statement-level ones and links related ones, LSdiff [36] summarises systematic changes as rules with exceptions, and refactoring-aware review tools show a detected refactoring [61] as a refactoring rather than as deleted and inserted lines [12, 28]. The options of everyday tools are visualisation devices over an unchanged line script: ignoring whitespace, highlighting changed words inside a line, colouring moved lines.

Pictures. For images the distinction is established. The mean squared error of two images is a property of their pixel delta, and Wang et al. [64] show that a measure of structural similarity agrees with human ratings of image quality better than it does. Yee's perceptual metric [67] decides whether two renderings differ *visibly*,

PREPRINT

for regression tests of rendering software, and pixelmatch [1], written to compare screenshots in tests, counts a pixel as changed only when its perceptual colour difference [39] passes a threshold and ignores anti-aliased pixels [62].

Containers. The diffoscope tool [53] of the Reproducible Builds project [41] recursively unpacks archives, transforms binary formats into a human-readable form and compares the results. It serves a setting in which the delta itself is the object of interest, because every byte that differs between two builds matters; a reviewer of a commit asks a different question, namely what was changed on purpose.

2.3 Definitions

Let A and B be the two versions of a file. A *content model* reads a version as a set of *elements*, $E(A)$, each of which owns a part of what the file holds: the lines of a text, the tokens and nodes of a syntax tree, the pixels of a picture, the members of an archive, the glyphs of a font, the pages of a document, the messages of a catalog.

Delta. A delta from A to B is a sequence δ of operations from a fixed set such that applying it to A yields B: $apply(\delta, A) = B$. Correctness is its only requirement. Among correct deltas, quality is a cost $c(\delta)$, the number of operations or the size of their encoding, and the best delta is a cheapest one.

Mapping. A mapping is a relation $M \subseteq E(A) \times E(B)$ that says which element of one version corresponds to which element of the other. An element of A that M leaves out is deleted and one of B is inserted; a pair whose two elements differ is an update, and a pair whose position among the other pairs differs is a move. A one-to-one mapping and a delta carry the same information up to the order of execution [15], so a mapping is a delta with its order forgotten, and every cost model on deltas ranks mappings as well. A mapping that a human writes down need not be one-to-one: Section 5 measures how often it is not.

Painting. A painting is what the reader of a diff sees. For text it is a pair of labellings $P = (p_A, p_B)$ that give every byte of each version either no label or one of *delete*, *insert*, *update* and *move*, with *delete* only in A and *insert* only in B. For content that is not text it is a set of regions or members with such labels together with a *verdict*, a statement of how much a viewer would see of the change (Section 3.4). A painting is not executable: it labels both versions and presupposes both.

Projection. A projection π_θ takes a mapping to a painting. Its parameter θ is the reader's preference, for which this paper uses two presets: *Minimal*, which paints only what changed, and *Full*, which also paints the brackets, separators, leading whitespace and whole identifiers that a change brings with it. The *literal* projection paints exactly the bytes owned by each deleted, inserted or updated element and nothing else; it is what a tool does that colours an edit script as it stands.

What each has to satisfy. Table 1 compares the two. A delta has to be correct and should be cheap, and nothing else is asked of it: many different cheapest scripts convert A into B, and to the machine that applies them they are interchangeable. A visualisation answers to a reader, and we state its requirements as four properties. *Faithfulness*: text that the painting leaves unmarked is unchanged and in place, up to what the reader's preference declares insignificant, such as indentation under *Minimal*. *Agreement*: the painting does not contradict the mapping it shows, so a token that the mapping pairs with an identical token is not painted as deleted on one side and inserted on the other. *Form*: what is marked follows the units a reader perceives, so a token is marked whole, a statement takes its terminator, and no highlight ends in trailing whitespace. *Preference*: the paintings for different readers are ordered and consistent, so *Full* marks everything *Minimal* marks, and the two never disagree about what survived. None of the four follows from correctness and minimal cost, and some contradict minimality: a rename painted as one whole identifier marks bytes that did not change. Section 3.5 turns the four properties into 31 checkable invariants, and Section 5 uses them to measure how far a delta is from a visualisation.

Table 1: A delta and a visualisation of the same change.

	Delta	Visualisation
Consumer	A machine, which executes it	A human, who reads it
Must be	Correct: applying it to one version yields the other	Faithful: what it leaves unmarked is unchanged for this reader
Judged by	Cost: the number of operations or encoded size	Agreement with what a human would mark
Unit	An operation on an element	A marked region of what is displayed
Executable	Yes	No: it presupposes both versions
Minimal	By definition	Not necessarily: a move, a whole renamed identifier
Depends on the reader	No	Yes: *Minimal* and *Full*
Text	Line edit script	Marked lines and words
Code	Tree edit script	Marked tokens, moves shown as moves
Pictures	Differing pixels	Changed regions, and how visible the change is
Containers	Differing bytes or members	Changed members, each shown by what it holds

Figure 2 shows the four accounts for the change of Figure 1. The line script is correct and marks three statements that did not change. The tree script is the cheapest correct one and marks five separate tokens, with the space between two of them left out. The two paintings follow the conventions of the ground truth of Section 3: the condition is one insertion, under *Full* together with the indentation of its line, and the three statements stay unmarked, because being indented further is all that happened to them.

2.4 Tools

The tools measured in Section 8 are listed here by the content they address.

Text. Unix `diff` is the gold standard. It treats each file as a sequence of lines, computes an edit script between the two sequences

DRAFT

LINE EDIT SCRIPT

```
def save(rec):
  if rec.dirty:
    db.begin()
    db.put(rec)
    db.commit()
```

TREE EDIT SCRIPT

```
def save(rec):
  if rec.dirty:
    db.begin()
    db.put(rec)
    db.commit()
```

MINIMAL PAINTING

```
def save(rec):
  if rec.dirty:
    db.begin()
    db.put(rec)
    db.commit()
```

FULL PAINTING

```
def save(rec):
  if rec.dirty:
    db.begin()
    db.put(rec)
    db.commit()
```

Figure 2: Four accounts of the change of Figure 1, on its after side; a shaded span is marked as inserted. The line script also marks three lines of the before side as deleted. The tree script inserts five tokens and moves the block, which costs nothing and is not shown.

with Myers' algorithm, and reports it as hunks of deleted and inserted whole lines. GNU `diff` caps the algorithm's cost with a heuristic by default, so on large inputs with many differences the script it reports can be longer than the shortest one. It has no move operation and no sub-line detail. **`git diff`** defaults to the `xdiff` engine, which offers four algorithms: Myers, the default; *minimal*, which spends more time to find a shortest script; *patience*, which anchors on lines unique to both sides before recursing; and *histogram*, a faster generalisation of patience. All four emit the same output form as Unix `diff`. **Neovim's diff mode** (`nvim -d`) runs the same `xdiff` line pass and then a second, character-level pass that highlights what changed inside each paired line. **BDiff** [43] reasons about blocks of lines rather than single lines and reports character ranges within changed lines.

Code. GumTree [23] is an AST diff. Instead of one global tree-edit-distance computation, it runs a top-down phase that greedily matches large isomorphic subtrees, then a bottom-up phase that matches remaining container nodes by a Dice coefficient of their already-matched descendants, and a recovery pass that pairs up deleted and inserted identical subtrees the first two phases missed. **difftastic** [32] and **diffsitter** [20] are a newer generation of tools built on tree-sitter [13] grammars. Both prioritise a diff a human reads comfortably over an exact node-to-node mapping: difftastic reduces the parse tree to an s-expression and aligns it with its own structural algorithm, and diffsitter applies a related alignment over a smaller, hand-picked set of compiled-in grammars. **srcDiff** [16] diffs the srcML markup of C, C++, C#, Java and Python source, detects moves, and writes one merged document in which the text of each side is marked as deleted, inserted or common.

Other content. Git judges a file binary by its bytes, chiefly by a NUL byte, and for a binary file prints that the two versions differ and nothing more. A user can configure a `textconv` driver per path pattern, a program that converts each side to text before the line diff runs. The conversion is lossy, and each format needs its own program and its own configuration [57]. diffoscope and pixelmatch, introduced in Section 2.2, are the two tools of this kind that OMNIDIFF builds on.

3 A Living Ground-Truth Dataset

This section describes the dataset the research questions are answered on: the project it belongs to, the corpus of repositories it is drawn from, how changes were sampled, what ground truth each content type carries, the invariants that hold the ground truth together, and how the dataset grows. No research question is answered here.

3.1 The project and the corpus

The dataset is the test corpus of OMNIDIFF, the tool of Section 9, which is developed in the open. On 10 November 2026 its repository, created in October 2025, had 1,313 GitHub stars and 35 forks. Developers star a repository to show appreciation and to bookmark it [10], so we report the count as a signal of interest, not of use. The tool is released as source on crates.io, as prebuilt binaries for Linux, macOS and Windows, through Homebrew, a signed apt repository and a Nix flake, and with editor integrations for VS Code and Neovim. Every fixture of the dataset is also a regression test of the tool, and a release is gated on the tool's accuracy over all of them. This is what makes the dataset a living one. It is not frozen for a paper and then left; it is the thing the project's daily development is measured against, and Section 3.6 shows how it has grown.

To understand the environment diffing tools are used in, we built a corpus of 7,045,754 files in 30 languages from two sets of open-source repositories:

- The Curated list consists of 100 open-source repositories, selected manually from several lists of the most popular repositories and extended by repositories we were already aware of. This biases the list towards active, widely used codebases, with good coverage of many programming languages. The Curated list is also the smaller list the project's automated test infrastructure runs over.
- The Full list consists of 7,444 repositories, created by expanding the Curated list with all repositories present in the Gentoo Linux distribution's package list. It includes codebases of widely different activity, code quality, user base and mix of languages.

We cloned each repository to a depth of 50 commits from every main branch. While we cannot guarantee that all repositories remain accessible, the list of repositories, the cloning script and all analysis are available from the paper's repository and can be replicated.

We fetched, parsed and measured the corpus on a single machine: an Intel Xeon E3-1275 v5 with 64 GB of RAM, running Linux 6.8. The checkouts and the measurement database sit on 4 16 TB WDC Ultrastar DC HC550 drives in RAID 5. We record this because some results are hardware-bound, notably the wall-clock budget we hold whole-tree APTED to in Section 7 and the per-tool timing of Section 8.

3.2 What the repositories hold

We categorised the files by extension and content. Only two thirds of all files are code (Figure 3). The rest is data, configuration, documentation, and a residual of unknown type, which limited manual inspection suggests is mostly data. A tool that diffs only code leaves a third of the files to something else.

DRAFT

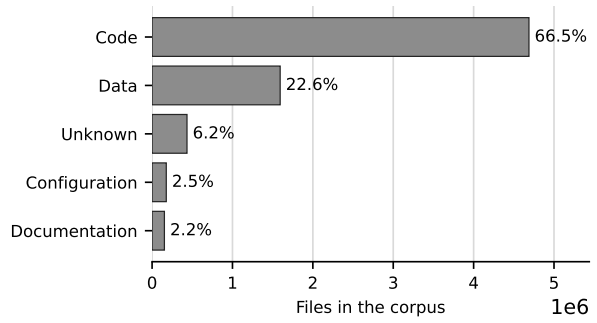


Figure 3: Distribution of file types across the corpus.

Restricted to code files, the top row of Figure 4 shows the distribution of bytes, lines of code and AST nodes per file. The median code file has 43 lines, the 99th percentile 2,511 and the largest 13,025,371. The node counts are over the 76.5% of code files that can be parsed (3,586,682 of 4,688,419); the rest are in a language without a grammar, flagged as generated and not parsed, or empty. Of those that parse, the median has 407 AST nodes, the 99th percentile 27,546 and the largest 23,584,040. Byte size and AST node count are strongly correlated for files within the 99th percentile of both, Pearson $r = 0.9156$, which simplifies to $|\text{AST}| \approx \text{bytes}/5$ for quick estimates.

3.3 Sampling changes

Code. From both the Curated and the Full list of repositories, we randomly sampled 10 diffs per language. That gave 240 diffs from the Curated list across 23 languages, and 267 from the Full list across 25 languages. The two language counts differ because the Curated list's repositories hold almost no R or Scala code, so those two languages are drawn from the Full list only. The Full list's count exceeds 10 per language because some languages were drawn a second time after their first rounds were largely rejected. Every sampled diff was manually inspected for sampling errors. At this stage, 25 diffs were rejected, mostly because language detection detected the wrong language: in particular TypeScript (7 rejected) and R (11) file extensions were used for other languages or data. This resulted in a dataset of 219 diffs from the Curated and 243 diffs from the Full list. As Section 4 shows, a pure random sample is heavily slanted towards small changes. To account for this, the sample was augmented by a stratified sample based on the lines of code of the files being changed. The strata were 1 to 10 lines, 11 to 30, 31 to 100, 101 to 300, 301 to 1000, 1001 to 3000 and more than 3000 lines of code, and the larger of the two sizes was used. In each stratum, 10 diffs were randomly sampled per language, and inspected as above.[2]

[2]The dataset and this paper are being written in parallel. Drawing and solving the stratified sample, and hand-annotating the Defects4J list below, are both still in progress, and the ambiguity pass over those two is not finished. Every figure in this paper is over the 1,699 changes solved so far, except the cost comparison of Section 5, the tool comparison of Section 8 and the line-level, speed and robustness figures of Section 9, which were measured at an earlier stage of the dataset, over the 1217 changes solved then, and, for OmniDiff's own rows in them, with an earlier version of the tool. Every figure that depends on the ambiguity pass is a lower bound for those two datasets.

A fourth list extends the dataset. The AST node-mapping oracle of Alikhanifard and Tsantalis [3] contains the 996 AST mappings for 800 bug-fixing changes of Defects4J [35]. The file contents its offsets index into come from Falleri and Martinez' replication package [22]. We solve those same 996 diffs, so that our mappings can be compared against an independently authored reference on a corpus that other AST-differencing evaluations already use.[2]

Other content. From the Full list we sampled changed pairs per content family: pictures, cursors, fonts, archives, compiled message catalogs and PDFs. Diffing several files at once and detecting moves across files are out of scope of this paper.

3.4 Ground truth

The ground truth of a change depends on what the file holds. Code has the deepest one, with two layers that correspond to the mapping and the painting of Section 2.3; other content has a verdict. The annotators are the authors, who each have at least 15 years of experience in software development and have reviewed tens of thousands of real-world code changes in industrial environments.

Mappings. Each code change was solved by hand. Solving a change means writing down, for every node of one AST, which node of the other AST it corresponds to, or that it has none. Where the annotator judged several nodes of one side interchangeable with several of the other, the correspondence is recorded as a *multi-map group*, a set of nodes on each side of which any consistent pairing is equally correct.

Paintings. For 1,439 of the changes the annotator additionally painted the rendering: the spans of text a reader should see marked as deleted, inserted, updated or moved, chosen independently of which nodes carry them. A painted change is one of three cases. In the first, it carries a *Minimal* and a *Full* painting, the annotator's finding that the change has a sparse and a generous correct rendering; a tool's sparse output is compared with the first and its generous output with the second. In the second, it carries a single painting that names no preset, the finding that the change has one right rendering, which every preset of a tool has to produce. In the third, it carries a single painting named for one preset, and only that preset is compared. Section 6 counts the cases.

Verdicts. Content other than code has no AST to map, so its ground truth answers a different question: how much did the change change? The annotators judged 194 of the sampled pairs on a six-level scale. Three levels are strengths of "nothing": *invisible*, where only bytes differ (metadata, a lossless re-encoding); *imperceptible*, visible only once a tool highlights it; and *artifacts*, visible but chosen by no one, as compression or resampling leaves it. Three are kinds of "something": *edited*, where a part changed deliberately and the rest stayed the same; *redrawn*, where the whole changed and is still a version of the same thing; and *replaced*, where the after side is not a version of the before side. An archive, a font, a PDF, a catalog or a cursor of several pictures is judged as a whole and per changed member, 484 verdicts in all.

Embedded code. Code also changes inside files of other languages. 4 of the 1,699 code changes edit code held in a string of the host language, in 3 embedded languages, and their ground truth maps and paints the embedded code's own AST as well as the host's.

DRAFT

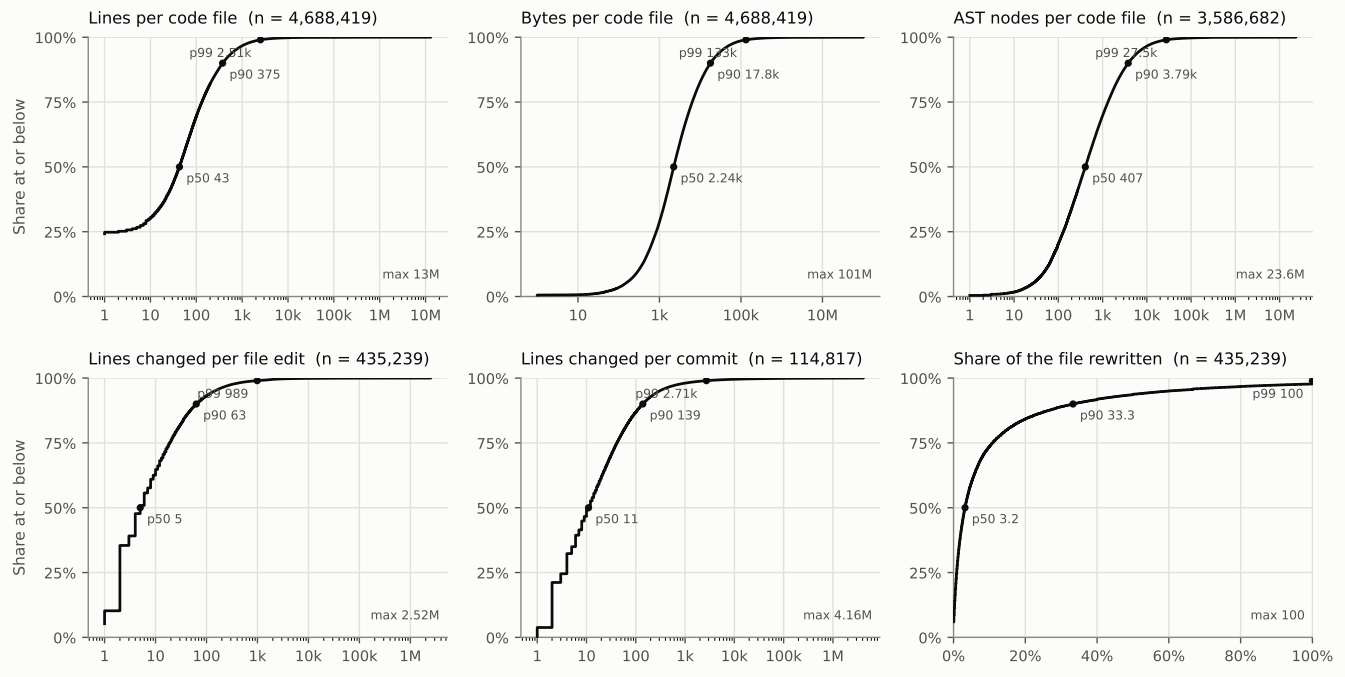


Figure 4: The shape of the corpus, every distribution drawn the same way: the share of the population at or below the value on the axis, with the median, 90th and 99th percentiles marked on each curve and the maximum in the corner. Top: size of every code file in the corpus, in lines and bytes, and in AST nodes for the code files that parse, which is why the third panel's *n* is smaller. Bottom: size of every code-file modification in the corpus's recent history, in lines changed per file, lines changed per commit, and the share of the file an edit rewrites. The horizontal axes are logarithmic except the last.

3.5 Invariants

A hand-made ground truth of this size is made by several people over months, and its answers drift: the same kind of change is painted one way in one fixture and another way in the next. We therefore state the conventions of the ground truth as *invariants*, properties that the mapping and the paintings of every fixture have to satisfy, each checked by a program that reads the ground truth alone and runs as a test of every fixture. There are 31 of them. They were added one at a time, as annotating and reviewing real changes turned up a contradiction inside a fixture or two fixtures answering one question differently, and each was decided by the annotators, not derived from a tool's output.

Table 2 shows the invariants in five families, which make the four properties of Section 2.3 concrete. Three rules concern the *form* of any painting: no highlight ends in a row's trailing whitespace, no two painted ranges claim the same byte, and a token such as `<=` or `private` is painted whole. Nine state what each *preset* paints and how the two relate: *Full* covers everything *Minimal* covers; a line that is inserted or deleted whole is painted whole under *Full*; *Minimal* never paints a line's indentation; a renamed identifier is painted at its differing words under *Minimal* and entire under *Full*; the space before an insertion inside a line goes with the insertion exactly when the insertion changed how many spaces are there; and the two presets never disagree about what survived. Ten require the painting to *agree* with the mapping: a leaf that the mapping pairs with an identical leaf is never painted deleted on one side and inserted on the other, a leaf the mapping deletes has at least one painted byte, and a byte painted as moved is never one the mapping leaves unmatched. Four keep *syntactic units* together: a `;` goes with its statement in the mapping and in the painting, a bracket with its partner, and the dot of a call with the member called. Five concern the *form of a mapping*, for example that two subtrees recorded as identical carry the same tokens. All but the 7 rules that read the mapping alone are rules about a painting, and none is needed for an edit script to be correct. We return to this in Section 5.

The invariants also repair the ground truth. When the set reached 31 rules, the ground truth of all fixtures, including the hand-written regression fixtures that this paper does not report on, broke them 435 times in 142 fixtures. We repaired 113 fixtures, each change being queued for review by an annotator, which left 111 violations in 44 fixtures. Over the 1,699 changes this paper reports on, 40 still carry a violation (98 in all), each recorded in the fixture as a known one, and 97.6% carry none. The remainder includes cases in which two rules ask for different things of the same bytes and cases that await a decision of the annotators.

3.6 Growth

The dataset grows in three ways. Sampling rounds over the corpus are drawn, inspected and solved, as described above. Hand-written fixtures pin one change pattern each; they are regression tests rather than a sample, and no figure in this paper includes them. Third, a misbehaviour that a user reports becomes a fixture with a

Table 2: The 31 ground-truth invariants by family, and how often three accounts of the same 1,437 changes break them: the hand paintings, the line edit script of `diff` painted as it stands, and the human tree mapping painted leaf by leaf as a tree edit script. *V* is violations and *C* the changes with at least one. The scripts are not judged by the 7 rules that read the mapping alone, which the last row leaves out for all three.

			Hand painting		Line script		Tree script	
Family	Rules	Example	V	C	V	C	V	C
Form of a painting	3	no highlight ends in trailing whitespace; <= is painted whole	0	0	60	21	126	14
What each preset paints	9	*Minimal* never paints indentation; *Full* paints a renamed identifier entire; the space before an insertion follows its count	35	16	9,019	1,102	18,605	1,052
Painting agrees with mapping	10	a leaf paired with an identical leaf is never painted deleted and inserted	37	16	4,299	889	23	12
Syntactic units stay together	4	a ; goes with its statement, a call's dot with its member	16	7	0	0	24	10
Form of a mapping	5	`NULL` → `nullptr` is one element changing, not a delete beside an insert	2	2	–	–	–	–
Rules that read a painting	24		80	31	13,378	1,272	18,778	1,056

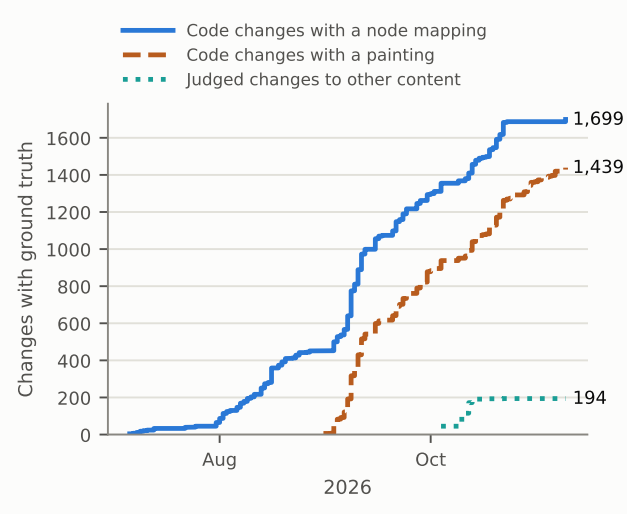


Figure 5: Growth of the ground truth, by the date each change's ground truth was first committed to the project's repository.

hand-made ground truth, so that the dataset accumulates the cases that users of the tool find hard.

Figure 5 and Table 3 show the growth, read from the history of the project's repository. The first hand mapping in the present format was committed in July 2026. In the 19 weeks since, ground truth was added or painted in 19 of them, at a median of 81 new changes per week and at most 432 in one week, to 1,699 mapped code changes, 1,439 painted ones and 194 judged changes to other content. The dates are those of the first commit of a fixture under its current name, so a renamed fixture is dated late.

The third way is the newest and the smallest. Of the 18 issues in the project's tracker on 10 November 2026, 10 were filed by people

Table 3: Ground truth added per month, by the date it was first committed. The four dataset columns count code changes with a node mapping; *Painted* counts code changes whose first painting was committed that month, *Content* judged changes to other content, and *Lang.* the languages the code changes cover by then.

Month	Cur.	Full	Strat.	D4J	Painted	Content	Lang.
Jul 2026	65	–	–	–	–	–	14
Aug 2026	155	232	–	–	6	–	24
Sep 2026	–	–	523	319	872	–	24
Oct 2026	–	–	268	125	485	194	24
Nov 2026	–	12	–	–	76	–	25
All	220	244	791	444	1,439	194	25

other than the authors; most of those ask for features of the viewer or for packaging, and two ask for a further content type. So far 1 of them has led to ground truth: a request for structural diffing of SQL led to SQL support in the tool and to 12 SQL changes, sampled from the corpus and solved by hand, which is the last row of Table 3 and the dataset's 25th language. The project's mapping viewer also lets a reader file a disagreement with one node of one fixture's mapping as an issue against that fixture, and two such issues are open. We report the count as it is: the dataset is open to growth from reports, and almost all of its growth so far is from sampling.

3.7 Cross-validation with existing datasets

The AST node-mapping oracle of Alikhanifard and Tsantalis [3] records, for 800 bug-fixing changes of Defects4J over 996 Java compilation units, which before-side node each after-side node corresponds to: 3,031,434 node correspondences in all, built by running six tools, inspecting every diff by hand against six stated criteria, and resolving disagreements by vote. It was authored by different

PREPRINT

Table 4: Changed files over the corpus's recent history, by what they hold. *Repos.* is the number of the 7,352 repositories with at least one such change.

Content	Formats	Changes	Repos.
Text	code, data, documentation	1,828,765	–
Pictures	PNG, JPEG, GIF, WebP, BMP, ICO, TIFF	45,761	725
Archives	zip (jar, Office, EPUB), tar, gzip, xz, bzip2	4,126	268
PDFs	PDF	1,482	105
Fonts	TrueType, OpenType, WOFF, WOFF2, EOT	77,859	84
Catalogs	gettext `.mo`, Qt `.qm`	1,038	41
Cursors	Windows `.cur`, `.ani`, X cursors, Hyprland `.hlc`	35,122	6
Other	binary, or failing to decode	23,857	–

people, for a different tool, on a different AST, before OmniDiff existed.

Their oracle uses Eclipse JDT nodes and ours tree-sitter nodes, so the two can be compared only where a record's character span is one some tree-sitter node also has. 78.0% of records resolve; the remainder are JDT constructs tree-sitter does not model separately, chiefly the elements inside a Javadoc comment, which tree-sitter reads as a single token.

The 444 compilation units we have solved so far are reported as a fourth dataset, Defects4J, beside the Curated, Full and Stratified samples throughout, and are included in every pooled total. Over the 440 of them for which the comparison has been run, our mapping and the oracle's agree to 99.62%: 624 disagreeing node pairs out of 84,887, and 67 of 14,059 at statement level.

4 Content of Real-World Changes

This section measures what a diffing tool is asked to show: which content commits change, and how large the changes are.

RQ1.1. What content do real-world changes touch?

Over the 50 most recent commits of each repository of the Full list, we counted every changed file, binary ones included: 2,021,166 changes. Git judges 9.1% of them (184,486) binary. We then classified by its bytes each of the 2,018,010 changed files that the commits' trees still hold, leaving out submodule entries; Section 9.3 describes the recognition.

Table 4 shows the result. The change counts are decided by a few repositories: fonts come from 84 repositories and cursors from 6, so the repository count is the one that says how common a format is. By it, pictures are the most widespread content other than text by far, changed in 725 repositories, followed by archives and PDFs. The *Other* row, 1.2% of changes, is mostly text in legacy encodings, compiled objects, audio, and EOT fonts compressed with MicroType Express. Within text, code is one kind among several: 31.5% of the files the commits touched were code files, and the rest of the text is data, configuration and documentation.

RA1.1 (Content)

90.6% of the 2,018,010 changed files hold text, of which code is one kind, and 8.2% hold pictures, archives, PDFs, fonts, message catalogs or cursors. Pictures change in 725 of the 7,352 repositories.

RQ1.2. How large are real-world changes?

We have size data for one content type, code, where size can be counted in lines. Across both repository lists we analysed 114,817 commits, at most 50 commits deep per repository, so that large repositories, notably the Linux kernel, do not skew the results. They hold 578,783 code-file edits, of which 75.2% were modifications; the remainder created or deleted a file and so present no pair of versions to compare. To allow for comparison with existing research, `git diff` was used to count changed lines.

The bottom row of Figure 4 shows the sizes of the edits. The median file edit changes 5 lines and the median commit 11; 64.8% of file edits and 49.1% of commits change ten lines or fewer, while the 99th percentiles are 989 and 2,712 lines. Our results are broadly aligned with existing research, among them Arafat and Riehle, who report that 47.5% of all commits touch 10 lines or fewer [4], with the long tail stretching to tens of thousands of lines. The median edit rewrites only 3.2% of the file it touches, and 59.9% of edits rewrite under 5%. In the 99th percentile, an edit rewrites the file entirely (100.0%).

RA1.2 (Size)

The median code-file edit changes 5 lines and rewrites 3.2% of its file; 64.8% of edits change ten lines or fewer, and at the 99th percentile an edit rewrites its file entirely.

A diffing tool therefore has to be good at two unrelated jobs: finding a handful of changed lines in an otherwise untouched file, which is the overwhelming majority of edits, and staying usable when nothing is left to anchor on. We did not measure the size of changes to other content, for which a line count has no meaning; the verdicts of Section 5 are the closest measure we have.

5 Delta versus Visualisation

Section 2 argues that a delta and a visualisation answer to different requirements. This section measures the distance between them in four ways: by the invariants a painted delta breaks, by how much of a delta reaches the reader, by the mappings no delta can express, and by the mappings a cost model ranks below the cheapest.

RQ2.1. How often does a delta, painted as it stands, break the rules that a visualisation obeys?

The invariants of Section 3.5 are rules of visualisation, so we hold three accounts of each change against them. The first is the hand painting itself. The second is the *line script*: the edit script of GNU `diff`, with every deleted line painted as deleted and every inserted line as inserted, from its first column to its last character. The third is the *tree script*: the human node mapping read as a tree edit script and painted by the literal projection of Section 2.3, that is, every deleted or inserted leaf painted whole and every leaf paired with a differently reading leaf painted as an update. A move is free in the

PREPRINT

cost model and the script does not mention it. The tree script is the most favourable delta we can construct, because its mapping is the human's own and so has no mapping error. A delta has no presets, so each script is filed as the *Minimal* and as the *Full* painting alike. The population is the 1,437 painted changes whose two files parse and that hold no embedded code, and each script is judged by the 24 invariants that read a painting.

The right-hand columns of Table 2 show the result. The line script breaks an invariant on 88.5% of the changes (1,272), 13,378 times in all. Most of its violations come from one of two sources. On 61.9% of the changes it marks as deleted and inserted, or as removed, a token that the human mapping keeps, which is the cost of a script without updates or moves. Most of the rest is indentation, which a line script always marks and *Minimal* never does. The tree script breaks an invariant on 73.5% of the changes (1,056), 18,778 times. It almost never contradicts its own mapping: it breaks an agreement rule on 0.8% of the changes. What it lacks is everything between the nodes: on 63.0% of the changes it breaks one of the four rules about which whitespace a painting covers, and most of the remainder are rules of granularity, such as a renamed identifier painted entire under both presets. The hand paintings break a painting rule on 2.2% of the changes.

RA2.1 (Invariants)

Painted as it stands, a line edit script breaks at least one of the 24 painting invariants on 88.5% of 1,437 changes, and a tree edit script built from the human-preferred mapping on 73.5%, against 2.2% for the hand paintings.

The two scripts fail differently, and the difference is the argument of this paper in small. The line script has the wrong correspondence, and no rendering can repair that. The tree script has the right correspondence and is still not a visualisation on 73.5% of the changes, because the correspondence is over nodes and the reader sees text: which whitespace, which separator and how much of a changed word to mark is decided by a projection, not by the mapping.

RQ2.2. How much of a delta reaches the reader?

For code, we count the nodes that can be displayed at all. A syntax tree contains two kinds of nodes. Some carry text of their own, such as an identifier, an operator, a literal or a comment. Others are pure structure: a `block` whose extent is exactly its children's, an `expression_statement` wrapping one expression, the chain of single-child nodes a grammar interposes between a statement and the name inside it. A rendered diff highlights regions of source text, so a structural node does not appear in the output on its own. We call a node visible if any byte inside its extent lies outside every one of its children. A tool that maps every displayed token correctly and every structural wrapper wrongly produces a diff that is correct, and it scores badly on a metric that counts all nodes equally.

For other content, we count the changes that have nothing to show. Every judged pair of Section 3.4 differs in its bytes, so each has a non-empty byte delta. Of the 194 pairs, the annotators judged 25 invisible, 11 imperceptible and 24 to hold only artifacts, 60 in all, and of the 290 changed members of containers that they judged separately, 94 (32.4%) fall in the same three levels. The remaining pairs are 102 edited, 12 redrawn and 20 replaced.

Table 5: Changes whose human-preferred mapping needs an $N{:}M$ site, per dataset of Section 3 and pooled.

Dataset	n	$N{:}M$ changes	Share
Curated	220	33	15.0%
Full	244	29	11.9%
Stratified*	791	47	5.9%
Defects4J*	444	63	14.2%
Curated + Full	464	62	13.4%
*All**	1699	172	10.1%

*The ambiguity pass over the Stratified and Defects4J changes is not finished, so these three figures are lower bounds: a change nobody has reviewed for ambiguity is recorded as having none. *Curated + Full* is the rate over the two datasets whose pass is complete, and is the figure RA2.3 states.

RA2.2 (What reaches the reader)

32.4% of the AST nodes in 1,699 code changes carry no text of their own and cannot appear in a rendered diff. Of 194 judged changes to other content, all of which differ in their bytes, 30.9% show a viewer no deliberate change.

The consequence for code is not that node-level metrics should be abandoned. It is that a rate over all nodes and a rate over visible nodes answer different questions, and any paper reporting one should state which. This paper reports both. The consequence for other content is that a tool which reports every pair with a non-empty delta as changed misreports 30.9% of them.

RQ2.3. What percentage of real-world code changes have a human-preferred node mapping that the common edit operations (Insert, Delete, Update, Move) cannot express?

The human solvers of Section 3.4 were not able to solve all changes with a one-to-one mapping. 62 of 464 solved changes (13.4%) require an $N{:}M$ mapping: one or more nodes from one side map to one or more nodes from the other.[3] That rate is over the 464 changes of the Curated and Full lists. The two lists agree within sampling noise, at 15.0% of 220 changes on the Curated list and 11.9% of 244 on the Full list (Table 5). Those two are the datasets whose ambiguity pass is finished, and the pooled rate RA2.3 states is over them. The table also carries the Stratified sample (5.9%) and the Defects4J changes (14.2%), and a rate over all 1699 changes, but all three are lower bounds: those two passes are still running, and a change nobody has reviewed for ambiguity is recorded as having none. The gap between 13.4% and 10.1% is the distance annotation still has to travel, not a difference between the datasets.

RA2.3 (Expressibility)

13.4% of code changes (62 of 464) have a human-preferred mapping that cannot be expressed at all using the common edit operations (Insert, Delete, Update, Move).

[3]The most common cause of an $N{:}M$ mapping is a refactoring, where two or more similar blocks of code are moved into a common function. The preferred solution is the one that maps all of them as moved.

DRAFT

This result bears on how tools are scored, because no established tool can produce these mappings. The node-level accuracy of Section 9 counts a group as correct whenever a tool's mapping is any of the pairings the group allows. The line-level comparison of Section 8 needs one answer per node, so it projects each group through a single representative pairing, members paired in source order; a tool that chose a different, equally correct pairing can be charged for the lines on which the two differ.

RQ2.4. What percentage of real-world code changes have a human-preferred mapping whose cost is higher than that of the cheapest mapping found?

Tree differencing algorithms look for the cheapest mapping, the one with the smallest number of operations needed to transform one tree into the other. We computed, for each change, the cost of the human mapping and the cost of a mapping found by a state-of-the-art matcher, under unit costs for insert, delete and update and a free move. Of the 1217 changes compared,[2] the two costs are equal 1057 times. In 29 (2.4%) the human solution is strictly costlier. The most common reason is that humans treat groups of tokens as one unit, not by their syntax but by higher-level logic. A common example is the `return` keyword: if one function was added and another deleted, matching the two `returns` lowers the tree edit distance, but a human finds the match confusing, because the keyword belongs to the logic of the function that contains it. Existing tools take notice of this: GumTree [23] does not match subtrees below a threshold size, to avoid such spurious mappings.

RA2.4 (Cost)

On at least 2.4% of code changes (29 of 1217), the human-authored mapping costs strictly more, under unit costs for insert, delete and update and a free move, than a mapping a tree-differencing algorithm found for the same change.

6 Uniqueness

A delta is one of many: any cheapest script will do. This section measures how often the human-preferred account of a change is not unique either, first in the mapping and then in its visualisation. Existing tools all provide a single diff. Line-based tools mark lines as inserted or deleted or match lines one to one, and tree-based tools match nodes one to one. No tool reports several solutions.

Manually examining real-world changes, we found that some have several equally correct solutions. Figure 6 is a motivating example. The AST of the before snippet contains one `boolean_operator` node. The after snippet contains two, one a direct child of the other. There is no objective way to determine which of the two is added and which one should be matched. The edit distance of both mappings is exactly equal, and since `boolean_operator` is an internal node with no rendering of its own, both also produce the same visualisation, so human judgement cannot break the tie either.

RQ3.1. What percentage of real-world code changes have multiple optimal node mappings?

We count a change as admitting several optimal mappings when its ground truth records at least one multi-map group (Section 3.4). Within a group, any one-to-one pairing that covers the smaller side is equally correct, and the nodes left over on the other side are

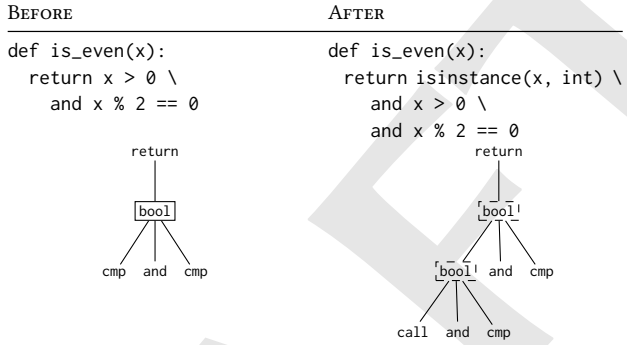


Figure 6: A change with no unique correct mapping. The before AST holds one `boolean_operator` node (`bool`, boxed) and the after AST two, one nested in the other (dashed); matching the before node to either costs the same and renders the same. `cmp` is a `comparison_operator`.

deleted or inserted. Across the dataset the annotators recorded 983 such groups.

Even without such groups, several solutions can exist. Where the annotator's mapping and a matcher's mapping for one change cost exactly the same under the unit cost model and still disagree about which nodes pair with which, the cost function cannot separate two different answers. That happens on 32 of the 1217 changes compared in Section 5 (2.6%).

RA3.1 (Mapping uniqueness)

In at least 13.4% of code changes (62 of 464), humans found that some set of nodes admits several equally correct pairings. Independently of that annotation, on 2.6% of changes the human and an algorithmic mapping tie on cost while disagreeing on the mapping, so the cost function alone cannot pick between them.

RQ3.2. What percentage of real-world code changes have multiple equally correct visualisations, and how do those differ?

Settling the mapping does not settle what the reader sees. Of the 1,439 painted changes, 899 (62.5%) carry a *Minimal* and a *Full* painting, the annotator's finding that a reader who wants only what changed and a reader who wants its surroundings marked are both right. 485 (33.7%) carry one painting that names no preset: for these a human considers the rendering settled, whatever the reader's preference. The remaining 54 changes carry only a *Minimal* painting and 1 only a *Full* one so far.

We compared the two paintings byte by byte on the 898 changes that carry both.[4] *Minimal* paints 229,104 bytes and *Full* 307,553, 1.34 times as many. *Full* never leaves unpainted a byte that *Minimal* paints (0 such bytes), which is one of the invariants, and the two give 1,317 bytes different labels. The two paintings are byte-identical on 1 of the changes, as expected: an annotator who would paint both alike records a single painting instead. Table 6 shows what the 78,449 bytes that only *Full* paints hold. 68.2% of them are whitespace,

[4] One change with two paintings is left out: its paintings are two alternatives that name no preset.

DRAFT

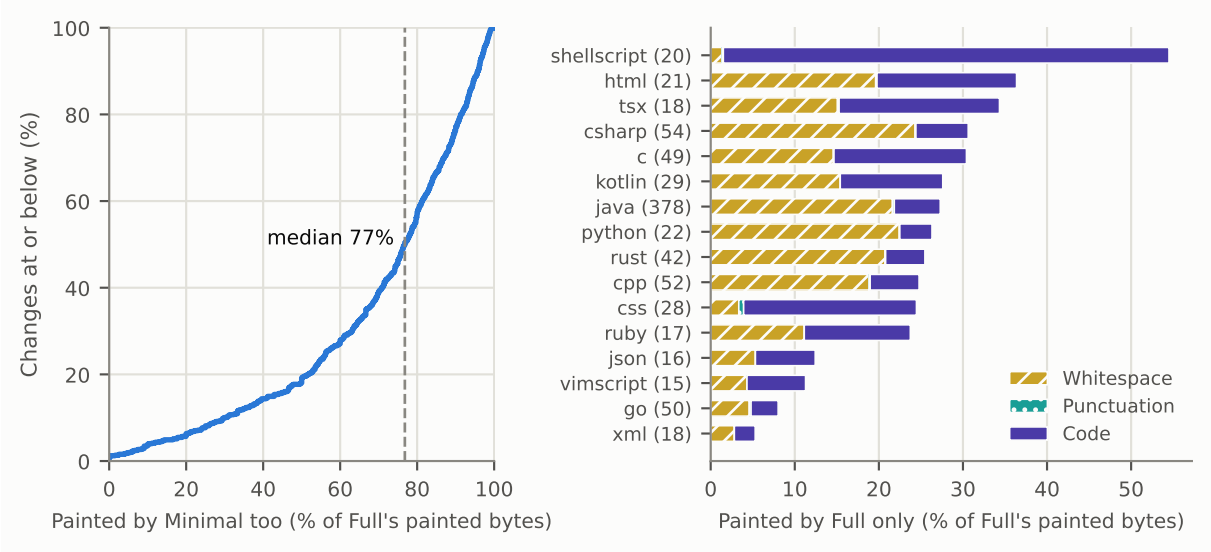


Figure 7: The *Full* hand painting against the *Minimal* one, over the changes painted both ways. Left: per change, the share of the bytes *Full* paints that *Minimal* paints as well, as a cumulative distribution. Right: per language with at least fifteen such changes (count in parentheses), the share of the bytes *Full* paints that *Minimal* leaves unpainted, by what those bytes hold.

Table 6: Bytes the *Full* painting paints and the *Minimal* painting of the same change leaves unpainted, by the label *Full* gives them and by what they hold, over the 898 changes painted both ways.

Label	Whitespace	Punctuation	Code	All
Insert	41,606	17	6	41,629
Delete	8,307	3	1	8,311
Update	113	0	6,504	6,617
Move	3,495	434	17,963	21,892
All	53,521	454	24,474	78,449
Share	68.2%	0.6%	31.2%	

almost all of it the indentation and spacing of inserted and deleted lines. 31.2% are code: the unchanged part of a renamed identifier or a changed number, which *Full* paints as part of the update, and code that moved, which *Full* marks as moved in more cases than *Minimal* does. Punctuation alone, the brackets and separators that a change brings with it, is 0.6%.

The difference is small for most changes and large for a few. The median change has 20 bytes that only *Full* paints, the 90th percentile 216 and the largest 4,532. The left panel of Figure 7 shows the distribution relative to the size of the painting: in the median change, *Minimal* paints 76.8% of the bytes *Full* paints. The right panel shows that the share differs by language, and that what is added differs as well: mostly whitespace in most languages, mostly code in shell scripts and CSS. We conjecture that in those two a changed word or value is short and the unchanged remainder that *Full* paints with it is long by comparison.

RA3.2 (Visualisation uniqueness)

Of the 1,439 painted changes, 899 (62.5%) admit a sparse and a generous correct rendering, and 485 (33.7%) have one rendering only. Where both exist, the generous one paints 1.34 times the bytes, and 68.2% of what it adds is whitespace.

For Section 2 this is the last piece of evidence: even the visualisation of a change is a family, with the reader's preference as its parameter, and the family has a single member for 33.7% of the painted changes. A delta has no such parameter.

7 Exact Deltas within an Interactive Budget

This section asks what the exact delta costs to compute. The data is for code, the content whose exact delta is expensive: a line script or a pixel difference is computed in time that is close to linear for typical inputs. The asymptotic complexity of APTED and Zhang–Shasha is $O(n^3)$ for general trees, where n is the number of nodes. APTED performs considerably better on real trees because it is less sensitive to the tree shape. Given this direction in the literature, we focus on source-code trees and ask the following question:

RQ4. What percentage of real-world source-code changes can a single, whole-tree tree-edit-distance computation complete within one second?

One second was chosen as a budget because it is where interaction stops feeling immediate. Nielsen's response-time limits [47] put one second at the boundary at which a user's flow of thought stays uninterrupted but the delay becomes noticeable; below about 100 ms a response reads as instantaneous. Practitioners rate the reaction time of review tooling as very important [40], and developer satisfaction with code review falls as changes wait longer [11].

Developers have had decades to get used to instant diffs from line-based tools, so any tool that expects to be adopted has to meet a similar bar.

We sampled 19,798 real (before, after) file pairs from single-parent commits in the corpus, stratified per language and per size bucket, with the buckets defined in Section 3.3: up to 142 pairs in each of the 168 cells of 24 languages and 7 buckets. 127 cells reached that target; the rest hold every pair the corpus had. On each pair, we ran our optimised APTED implementation, with a one-second budget enforced by operating-system limits.

The left panel of Figure 8 shows, for each artifact category, the share of pairs completed as a function of the time allowed. Size decides the outcome: within one second the computation finishes for 97.7% of code file pairs at 31 to 100 lines and for 46.1% at 101 to 300. The shape of the input matters as well: scripting languages complete more often than code, and data files less often.

RA4 (Exactness)

Within one second, a whole-tree tree-edit-distance computation finds the delta for only 46.8% of code file pairs.

8 State of the Art

This section measures what established tools do with real changes: what they show for each type of content, and, for code, how accurately and how fast.

RQ5.1. What do established tools show for each type of content?

For text, `diff` and `git diff` show a line script, and they do so for every text file, whatever it holds. For the 9.1% of changed files that git judges binary (RA1.1), git shows that the two versions differ and nothing else, unless a user has configured a converter for the format. The tools that go further each cover a part of the content. The AST-aware tools read code in the languages they have a grammar for: of the 1217 code changes of the comparison below, diffsitter's compiled-in grammar set processes 891, GumTree's generator set 1005, difftastic's 1182, and srcDiff, which parses five languages, 519. diffoscope unpacks containers and compares what is inside them as text or as bytes, and pixelmatch counts the pixels of two pictures that differ visibly. None of the tools we know of states for a changed picture, font or archive how much of the change a viewer would see, which is what the verdicts of Section 3.4 record, so we could score no established tool against them.

RA5.1 (Coverage)

Git shows a line script for the 90.9% of changed files it reads as text and only a notice for the 9.1% it judges binary. Of four AST-aware tools, the broadest reads 1182 of 1217 sampled code changes and the narrowest 519.

RQ5.2. How accurately do current state-of-the-art code diffing tools find the human-preferred visualisation?

RQ5.3. How fast are they?

Comparison tools. We measure eight established tools, all introduced in Section 2.4, on 1217 of the code changes of Section 3.[2] `git diff` is measured under each of its four algorithms, Myers, minimal, patience and histogram, giving eleven configurations in all. Five configurations report at line granularity only: Unix `diff` and the four `git` algorithms. Six report sub-line detail: BDiff [43], Neovim's diff mode, and the four AST-aware tools, GumTree (v4.0.0-beta8) [23], difftastic [32], diffsitter [20] and srcDiff [16]. Our own tool's figures are reported in Section 9, on the same basis and against the same ground truth.

Line-level agreement. None of the eleven configurations reports a node mapping or a painting in a form that can be compared directly, so we compare them on a common ground: which source lines each marks as touched, against the lines the human mapping touches. Because the tools cover different subsets of the changes (RA5.1), percentages are used.

We report agreement per change rather than pooled over lines in Table 7, and Figure 9 splits the same reading by the dataset each change was sampled from. *Perfect* means zero mismatched lines. ≥99% is "a developer has to look twice"; 95–99% is "a developer has to check". Below 95% the tool is no longer usable.

RA5.2 (Tool accuracy)

srcDiff renders 79% of the code changes it parses perfectly, but it parses five languages; no tool that parses most of the changes renders much more than three fifths of them perfectly, the best being difftastic at 64%.

The split by dataset in Figure 9 orders the datasets the same way for every configuration: each scores its own best on either the Stratified sample or the Defects4J changes, and all five line-granularity configurations do best on Defects4J. Bug fixes are short, localised edits, which is the shape a line-based diff recovers most reliably, so the ordering is a property of the datasets rather than of any tool.

Cost. Table 8 shows wall-clock percentiles, sorted fastest to slowest by median. Each tool is measured over the changes it supports rather than over one common subset, so the populations behind these rows differ: the text-based tools draw on every change, while an AST tool draws only on the languages its grammars cover. Two tools are reported twice: GumTree once invoked as a fresh process per change, the realistic cost of a single command-line invocation, and once run inside one persistent JVM across all changes, isolating JVM startup from GumTree's own matching cost; and BDiff likewise per process and inside one persistent Python interpreter, for the same reason.

RA5.3 (Tool cost)

The eleven configurations span more than two orders of magnitude at the median, from Unix `diff`'s 1.9 ms to GumTree's 534.9 ms per invocation. At the 99th percentile the same range runs from 4.1 ms to 6570.9 ms, three orders of magnitude apart.

We report both a cold and a warm figure for GumTree and BDiff because they answer different questions: the cold number is what a developer running a diff from the command line waits for, and the warm number is the matching cost once process startup is amortised away. Every timing number in this paper is the result of repeated runs rather than a single measurement. Each tool is run 3 times on every change, and the percentiles are taken over every

DRAFT

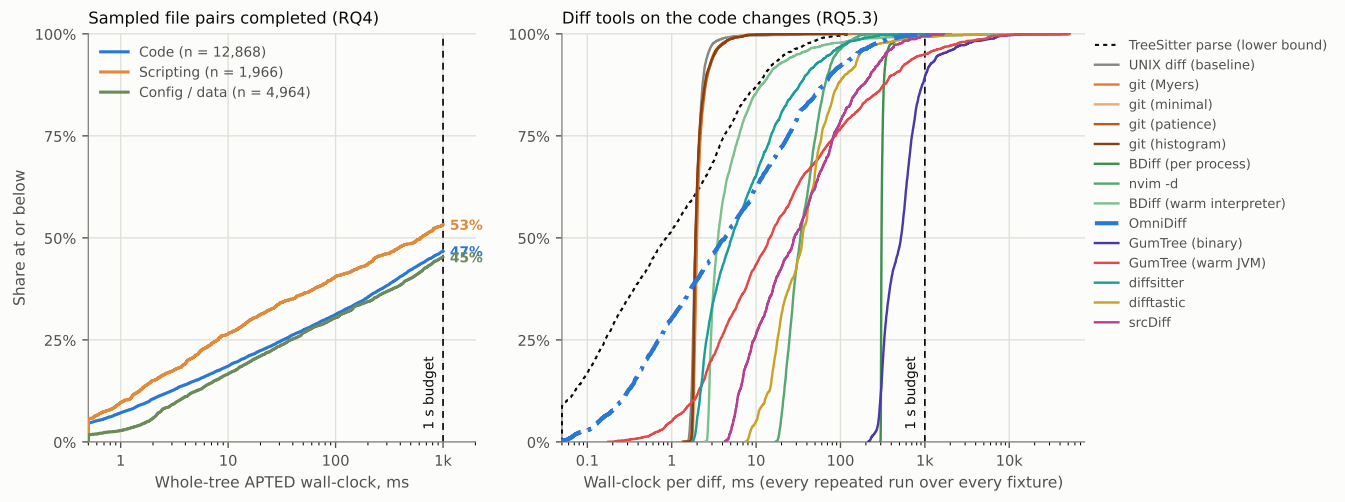


Figure 8: The one-second budget, read the same way twice. Left: the share of sampled file pairs whose whole-tree APTED computation completes within the time on the axis, per artifact category; the height of each curve at the dashed line is RA4, and the pairs the budget killed are the gap to 100%. Right: wall-clock per diff for every tool configuration of Section 8, every repeated run over every fixture, with OmniDiff charged for its parse and drawn dash-dotted; the dotted curve is the tree-sitter parse alone, the floor every AST-aware tool pays first. The dashed line is the same one second in both panels.

Table 7: Per-fixture line-level agreement with the human mapping, bucketed, for all eleven configurations. The upper block holds the tools that report whole lines only; the lower block holds the tools whose output carries sub-line detail, scored on the same line-level basis so that the eleven are comparable. "Perfect" means zero mismatched lines, not a rounded 100%. Each tool is scored on its own applicable subset (n), so percentages, not counts, are comparable across rows.

Tool	n	Perfect	≥99%	95–99%	<95%
Line granularity: line-only tools					
Unix `diff`	1217	713 (59%)	249 (20%)	160 (13%)	95 (8%)
`git` (Myers)	1217	718 (59%)	248 (20%)	159 (13%)	92 (8%)
`git` (minimal)	1217	718 (59%)	248 (20%)	159 (13%)	92 (8%)
`git` (patience)	1217	717 (59%)	247 (20%)	160 (13%)	93 (8%)
`git` (histogram)	1217	717 (59%)	247 (20%)	161 (13%)	92 (8%)
Line granularity: tools reporting sub-line detail					
BDiff (cold, per-invocation)	1217	688 (57%)	254 (21%)	179 (15%)	96 (8%)
`nvim -d`	1217	728 (60%)	247 (20%)	156 (13%)	86 (7%)
diffsitter	891	482 (54%)	204 (23%)	115 (13%)	90 (10%)
difftastic	1182	755 (64%)	197 (17%)	135 (11%)	95 (8%)
GumTree (cold, per-invocation)	1005	621 (62%)	158 (16%)	128 (13%)	98 (10%)
srcDiff	519	409 (79%)	68 (13%)	32 (6%)	10 (2%)

individual run pooled across changes rather than over per-change means: the repeat count is uniform, so no change is over-weighted.

9 OmniDiff

This section is the paper's tool contribution. OmniDiff diffs a file by what it holds. It recognises the content of a pair from its bytes (Section 9.1), diffs code with a grammar by mapping its AST and projecting the mapping to a painting, and any other text line by line (Section 9.2), and diffs binary content by one of two content engines (Section 9.3). Table 9 shows what each empirical result decided in its design, and Section 9.4 evaluates the tool against the same ground truth and on the same basis as the tools of Section 8.

OmniDiff has two targets:

- **Robust:** OmniDiff must handle every file up to the measured maximum, 23,584,040 AST nodes, with headroom above that number.
- **Fast:** OmniDiff must compute a diff in under 1000 ms for 99% of real-world changes, RQ4's one second applied to the population of real changes rather than to a sample of hard cases.



Figure 9: The line-level agreement of Table 7, split by the dataset of Section 3 each fixture was drawn from, with the pooled reading on the right. Each bar is one configuration over the fixtures it supports (n beside the bar); the segments are the four agreement buckets, and the Perfect segment carries its share.

Table 8: Wall-clock time percentiles for the eleven tool configurations, milliseconds, sorted fastest to slowest by median, with OmniDiff on the last row. *Max* is the slowest single one of the 3 runs over any fixture. Every configuration is timed as a whole process except OmniDiff, which is timed in-process and so pays no process startup; see Section 10.

Tool	p50	p90	p99	Max
`git` (histogram)	1.9	2.6	5.3	118.8
`git` (minimal)	1.9	2.6	5.2	38.0
Unix `diff`	1.9	2.4	4.1	246.1
`git` (patience)	1.9	2.7	5.5	35.9
`git` (Myers)	2.0	2.7	5.4	137.6
BDiff (warm interpreter)	3.6	14.0	277.1	41747.6
diffsitter	5.4	38.4	188.9	511.9
GumTree (warm JVM)	14.0	434.5	5389.2	12200.9
srcDiff	31.0	230.0	818.2	1682.1
`nvim -d`	33.6	70.9	150.4	2039.3
difftastic	35.0	113.8	637.4	4068.6
BDiff (cold, per-invocation)	306.5	330.8	605.1	42611.0
GumTree (cold, per-invocation)	534.9	1027.3	6570.9	13740.2
OmniDiff	4.6	77.4	420.1	1337.4

9.1 Recognising content

A content kind is recognised from the file's bytes, never from its name: under `git diff` both sides are temporary files, and an X cursor is an extensionless file named after the cursor. A pair is diffed as one kind or not at all, and a pair that does not decode is reported as any other binary pair rather than as an error, so that, run as git's external diff, one bad file does not abandon every file after it. Text needs no such recognition, but two kinds of it are widened: XML formats such as SVG are parsed with the XML grammar, and text in UTF-16 or UTF-32 with a byte order mark is decoded and diffed as text rather than as binary. Text in one of 26 languages with a tree-sitter grammar [13] is diffed as code; any other text is diffed line by line, with the changed part of a paired line marked.

9.2 Text and code

For code, OmniDiff first computes a mapping and then projects it to a painting, the two steps of Section 2.3. It matches two ASTs in five phases. Each phase either resolves part of the mapping directly or narrows the residual left for the next phase. Later phases only ever see nodes earlier phases left unmatched. A sixth step closes the mapping, recording the deletion or insertion implied by whatever every phase above left undecided.

Phase 1, hash-based descent. OmniDiff matches subtrees by two hash variants, in order: byte-identical subtrees, then same-shape subtrees with any leaf value. This phase is motivated directly by RA1.2: since most commits touch a small fraction of a file's lines, most of a typical file's AST is byte-identical between before and after. Resolving that identical majority with cheap hashing, before any tree-edit-distance computation runs at all, avoids paying $O(n^3)$ on content that never changed.

Phase 2, contextual exact matching. Comments and modifiers (attributes, decorators) that immediately precede an already-matched node, and diagnostic statements (logging calls, assertions, panics) that are byte-identical on both sides, get matched directly. Neither is reachable by phase 1's structural hashing: comments and modifiers are not part of the hashed AST shape, and diagnostic statements are held back deliberately, so that matching one does not fragment a larger match around it. These are the preferences RA2.4 measures: each of them can cost more than the cheapest mapping.

Phase 3, syntax-aware subtree matching. This phase generalises three matching strategies: matching by fully-resolved name (handling overloads and duplicate names across scopes), a greedy cost-estimate matcher for containers with no name structure, such as an `if` block or a loop body, and an overlap matcher pairing import statements that share a base path. Before any of these run,

Table 9: What each empirical result decided in OmniDiff's design.

Finding	Consequence
9.1% of changed files are binary to git, and pictures change in 725 repositories (RA1.1)	Content is recognised by its bytes and diffed by what it holds
The median edit rewrites 3.2% of its file (RA1.2)	Phase 1 settles the unchanged majority by hashing before any edit-distance computation runs
A tree script with the right mapping breaks a painting invariant on 73.5% of changes (RA2.1)	The renderer is a projection with rules of its own, not a colouring of the mapping
32.4% of nodes never reach the screen, and 30.9% of content changes show nothing deliberate (RA2.2)	Accuracy is reported on visible nodes and painted bytes; every content change is named by how much of it a viewer sees
13.4% of code changes have no expressible optimum and 2.6% tie at different mappings (RA2.3, RA3.1)	The phase order is the tie-break: the earliest phase to claim a node wins, and the closing step marks what no phase claimed
On 2.4% of changes the human mapping costs more than the cheapest one (RA2.4)	Phases 2 and 3 encode reader preferences that cost more than the optimum: comments and modifiers follow their node, names beat cost, diagnostics are held back
62.5% of painted changes have a sparse and a generous rendering (RA3.2)	Two render presets, *Minimal* and *Full*, over one mapping
Whole-tree APTED finishes within one second for only 46.8% of code pairs (RA4)	APTED runs only inside one already-matched pair (Phase 3), never over the whole tree
No established tool that parses most code changes renders more than 64% of them perfectly (RA5.2)	The bar the evaluation has to clear, on the same basis
Files reach 23,584,040 nodes (Section 3.2)	The Robust target below

large flat subtrees, such as a macro's token list or a big flat argument list, are pre-matched directly with Myers' algorithm [46], since a flat sequence has no tree structure worth exploiting. Each accepted pair is then handed to APTED [50], scoped to that pair alone, to compute its exact internal edit script. This is RA4 applied: the exact computation is affordable inside one matched pair, and only there.

Phase 4, residual alignment. Whatever remains unmatched after phases 1 through 3 is resolved by a Myers-LCS alignment [46] over the residual forest, bracketed by two passes of strict bottom-up propagation: a node whose children all match the children of exactly one node on the other side is matched to it, which shrinks the residual, and is run again afterward so that pockets the alignment just resolved can still bubble up to their now-complete ancestors.

Phase 5, move-detection recovery. The final matching pass. Ordered tree edit distance cannot express a subtree that relocated across a matched boundary as anything but a delete plus an insert. This phase, adapted directly from GumTree's recovery-mappings phase [23], pairs up whole subtrees left fully deleted on one side and fully inserted on the other, when they are byte-identical and large enough to rule out coincidence.

Projection. The renderer turns the mapping into a painting under one of two presets, *Minimal* and *Full*, the two the ground truth is painted in. It is not the literal projection: following RA2.1, it decides separately which whitespace, separators and brackets go with a change, whether a relocated node is shown as moved or left unmarked as a pure reindent, and how much of a changed word to mark. Its rules follow the painting invariants of Section 3.5, and where they can the two share code: the list of tokens that are painted whole is read by the renderer and by the invariant alike.

Code inside strings. After the five phases, every matched pair of host strings that changed is examined for code of another language. Where the string's context names a language (a sibling `"path"` naming a file, a YAML `run:` key, a `css` template tag) and both sides parse without an error, the decoded strings are diffed by the whole pipeline as files of their own, and the result is painted back onto the host file through a map from each decoded byte to its source. A changed string whose language is not detected is still diffed as text, line by line, so two edits far apart in one long string are painted apart rather than as one span from the first change to the last.

9.3 Other content

Two engines. A picture is pixels. Which pixels changed is decided by pixelmatch's perceptual test [1], so that re-encoding noise is not reported as change; changed pixels close to each other are grouped into one region, and the frames of an animation are aligned the way a text diff aligns lines. Everything else is a container of named members: an archive's files, a font's glyphs, a PDF's pages, a catalog's messages, a cursor theme's sizes. Members match by key and are hashed by what they show, so only a changed member is decoded, and it is diffed by what it holds in turn, a picture in a zip as a picture. A glyph is keyed by the character it draws and hashed by its outline and advance width, so a font re-hinted or re-wrapped as WOFF2 has no changed glyph. A PDF page is keyed by its number and drawn as a picture, a message by what it translates, and a Java class file in a jar is listed as its fields and methods and what each method's bytecode refers to, which diffs as text.

Verdicts. Each change, and each changed member of a container, is named by one of the six levels of Section 3.4. This is RA2.2 applied to content: the byte delta of such a pair is never empty, and a diff that reports a lossless re-encoding the way it reports a redrawn icon hides the one among the other.

9.4 Evaluation

Coverage of content. Of the 2,018,010 changed files of RA1.1, OmniDiff diffs 98.8% as text or by their content, against the 90.9% git diffs as text. Counting at most 1,000 changes per repository, so that a few very large repositories do not decide it, the share is 98.2%, and 6,481 of the 7,352 repositories are diffed entirely.

Code: mapping. Against the human mapping, OmniDiff maps 99.96% of AST nodes correctly (7,668 mismatches in 18,638,860), and 99.96% of the 67.6% of nodes that carry text and can reach the

DRAFT

Table 10: OmniDiff against the code ground truth, per change. *Exact*: no visible node mapped differently from the human mapping, or no byte painted differently from the hand painting. *Within 1%*: at most 1% of the visible nodes, or of the bytes of the two files. *Human mapping*: exact when the renderer is given the human mapping.

Compared	*n*	Exact	Within 1%	Human mapping
Node mapping	1,699	84.0%	96.8%	–
Painting, *Minimal*	953	58.9%	90.2%	62.3%
Painting, *Full*	900	52.4%	85.8%	53.6%
Painting, one rendering	485	84.7%	97.1%	85.8%

screen (5,087 in 12,595,261). Per change, Table 10 shows that it maps every visible node as the human did on 84.0% of the 1,699 changes (1,427).

Code: painting. The painting is compared byte by byte with the hand painting, in the three cases of Section 3.4; where a change has one rendering, both presets have to produce it, and a byte counts as wrong when either does not. Table 10 shows the result. OmniDiff paints every byte as the human did on 58.9% of the changes under *Minimal*, on 52.4% under *Full*, and on 84.7% of the changes with one rendering. The last column separates the two steps: given the human mapping instead of its own, the renderer is exact on 62.3%, 53.6% and 85.8%. Most of what is left is therefore the projection's and not the matcher's, and it is largest where RA3.2 puts the reader's preference: in what *Full* adds.

Code: invariants. We also hold OmniDiff's output against the 15 invariants that can be checked on a painting's labels alone, on the 1,437 changes of RA2.1. Its rendering of its own mapping breaks one of them on 17.7% of the changes, and its rendering of the human mapping on 9.7%. By the same rules the tree script, which is that same human mapping under the literal projection, breaks one on 67.2%, the line script on 88.4% and the hand paintings on 1.9%. The projection thus closes most of the distance between a correct mapping and a visualisation, and the matcher's errors account for the difference between the first two figures.

Code: comparison with established tools. On the line-level basis of Section 8 and its 1217 changes,[2] OmniDiff renders 90% of the changes perfectly, against 64% for the best established tool that parses most of them, and it completes every one. It marks 3,025 lines differently from the human mapping, a rate of 0.221%, the lowest of any tool measured. On the 519 changes srcDiff parses, it renders 89% perfectly against srcDiff's 79%. Table 11 splits the result by dataset. A line is a coarser unit than a painted byte, which is why this figure is higher than those of Table 10.

Code: robustness. Run over the 1,217 fixtures with a 120-second budget per fixture and no cap on tree size, OmniDiff completes every one. The largest fixture holds 198,406 AST nodes per side, the slowest takes 1236.9 ms, and peak heap use is 56.6 MB. The same measurement over the whole Full corpus reaches the files the fixtures do not. Every modified code file in the corpus's recent history (Section 4), 442,530 pairs across 25 languages, was diffed with no node cap under a 120-second budget and a 6 GB memory cap per process. OmniDiff completes 99.95% of them with 0 panics, at a median of 12.2 ms and a 99th percentile of 2,095 ms; the largest

Table 11: OmniDiff against the human mapping, in the four agreement buckets of Table 7 and on the same line-level basis, so that a row here can be read directly against a row there. *Perfect* means literally zero mismatched lines.

Dataset	*n*	Perfect	≥99%	95–99%	<95%
Curated	220	190 (86%)	20 (9%)	5 (2%)	5 (2%)
Full	232	194 (84%)	23 (10%)	9 (4%)	6 (3%)
Stratified	491	464 (95%)	8 (2%)	7 (1%)	12 (2%)
Defects4J	274	246 (90%)	15 (5%)	12 (4%)	1 (0%)
All	1217	1094 (90%)	66 (5%)	33 (3%)	24 (2%)

Table 12: Speed and accuracy per content family. Speed is milliseconds per sampled pair, decoding included; accuracy is against the human verdicts on the judged changes.

	Speed				Accuracy		
Content	*n*	p50	p90	p99	*n*	Perfect	Near
Pictures	230	2.6	83.4	887	131	69%	91%
Archives	100	3.5	405	3,857	23	87%	87%
PDFs	100	158	874	2,241	4	100%	100%
Fonts	100	8.6	119	2,042	11	82%	100%
Catalogs	81	0.5	1.5	7.4	12	83%	83%
Cursors	22	0.7	1.4	23.9	13	85%	100%
All	633	5.0			194	75%	91%

completed pair holds 7,092,498 AST nodes on each side, against the 23,584,040 of the largest file in the corpus that the Robust target names. The remainder is 96 pairs past the budget and 112 past the memory cap. The killed pairs are commits of 20 distinct files, all but one of them generated parser tables, minified bundles or embedded binary data; the exception is one commit of a 40,000-line single-header C++ library. Given 24 GB and a five-minute budget, one commit of each memory-killed file was tried again: 11 completed, one needed more memory still, and the rest ran past even that five-minute budget.

Code: speed. Over the 1217 changes of the tool comparison, OmniDiff's median wall-clock time is 4.6 ms, its 90th percentile 77.4 ms and its 99th percentile 420.1 ms, parse included. The right panel of Figure 8 places that curve among the eleven configurations of Table 8: at the median the line-based tools sit to its left and the AST-aware tools to its right, and besides diffsitter, over the 891 changes its grammars cover, it is the only AST-aware curve that reaches 100% before the budget line.

Other content: speed. Table 12 shows speed and accuracy per content family. Speed is timed over 633 pairs sampled from the Full list, on the machine of Section 3.1, one diff per pair from the bytes in memory, decoding included: unpacking an archive, drawing a font's glyphs or a PDF's pages. Pictures, catalogs and cursors meet the Fast target, with 99% of pictures under one second. Archives, fonts and PDFs do not: 94% of archives, 97% of fonts and 93% of PDFs finish within one second, because each changed member is decoded in turn and a font or a PDF draws every changed glyph or page. Over all families, 97.2% of pairs finish within one second, against the 99% the target asks.

DRAFT

Other content: accuracy. OmniDiff's verdicts equal every human verdict on 75% of the 194 judged changes (145), and are all within one level on 91%. Pictures, the largest set, are the weakest, at 69% perfect. We conjecture that most of that gap lies in the picture engine's thresholds between imperceptible, artifacts, edited and redrawn, which are set by hand and not yet fitted to the judged pictures. The PDF figure rests on 4 changes.

10 Threats to Validity

Construct validity. The invariants were written by the people who made the paintings, and the paintings were then repaired to obey them (Section 3.5). The low rate at which the hand paintings break them is therefore partly by construction, and RA2.1 measures the distance of a script from this convention of visualisation, not from every possible one. Two choices in that measurement favour or disfavour the scripts and are stated with it: the tree script is given the human mapping, which no algorithm produces, and both scripts are filed under both presets, so that the line script is charged for indentation under *Minimal*. The share of changes on which the line script marks surviving code as removed does not depend on the second choice.

Every rate this paper reports about ambiguity is a floor rather than an estimate. A multi-map group exists where an annotator noticed that two pairings were equally correct and recorded it; a change whose alternative mapping nobody saw is filed as unique. The same holds for the paintings: a change with a single painting is one for which the annotator saw one rendering. RA3.1's second reading, cost ties at differing mappings, needs no annotation to detect and exists for that reason. We score RA2.3 and RA3.1 over the 464 changes whose ambiguity pass is complete rather than over all 1,699. That pass covers the Curated and Full lists in full, so for those two lists the bound is one on annotator attention rather than one on coverage. The Stratified sample (47 of 791 changes) runs far under the rate the two finished lists show, which is an annotation gap rather than a finding about those changes; Table 5 prints its row and the *All* row under that caveat.

Because no whole-file exact computation is affordable at these sizes (RA4), RA2.4 compares the cost of the human mapping to that of a heuristic computation rather than to a proven optimum. Its rate is therefore a lower bound.

Section 8 scores tools at the line level because that is the only granularity all eleven configurations share. A line-level projection of a node mapping discards detail that the AST-aware tools compute and the line-based ones never had, which if anything favours the line-based tools. It also means the paintings never enter that comparison, so RA3.2's error term sits in those tables unremoved.

The content ground truth is a verdict, not a painting. A matching verdict says that OmniDiff judged how much a change changed as the annotator did, not that it found where: the regions and members it reports are not scored, except that a member the annotator judged has to be among them. The six levels are also coarser than a node mapping, so a perfect content verdict and a perfect code rendering are not the same achievement.

Internal validity. OmniDiff is timed in-process while every external tool is timed as a whole process, so it pays no process-startup cost that the others pay in full. We state this rather than correct for it, and report GumTree and BDiff both cold and warm so that a reader can see how large the effect is for the two tools where it dominates. Per-tool mean runtimes are additionally distorted by run order, whichever tool runs first per change absorbing cold-cache cost; to minimise this effect we quote percentiles rather than means throughout, and every timing number is the result of repeated runs.

The parts of this paper were not all measured at one state of the dataset and the tool. The tool comparison, the cost comparison and OmniDiff's line-level, speed and robustness figures are over the 1217 changes solved at an earlier stage and, for OmniDiff, an earlier version; the node-level, painting-level and invariant figures are over the 1,699 changes and the current version (footnote 2). We keep the two apart in the text rather than mix them in one table.

The ground truth is authored by the same people who built OmniDiff, which is a real risk of bias in its favour. Three things limit it: the mappings are authored against the change itself in a dedicated annotation tool with no tool output shown, the full annotation history is published with the dataset so that any individual mapping can be inspected and disputed, and Section 3.7 compares our annotation against ground truth authored by other people, for a different tool, before this one existed. That last check is the only one of the three that does not depend on our own discipline, and it is the reason the dataset was extended with Defects4J at all. The paintings and the content verdicts have no such check: they are made by the same people, and we know of no external ground truth for how a code change should be painted or for how much a change to a picture, a font or an archive changed.

The content timings are a single run per pair, where every code timing is repeated, so a content percentile carries the noise of one measurement.

External validity. The fixtures are deliberately not a fully random sample of real commits. They concentrate hard, structurally interesting changes, because easy ones exercise nothing worth measuring, so every tail figure in this paper is a stress reading rather than an estimate of what a developer waits for on a typical commit. Every accuracy result is also reported per dataset (Figure 9, Table 11), so a reader can separate the two random samples, the stratified one, and the Defects4J bug fixes from each other. The Defects4J units solved so far are how far annotation has reached, not a draw, and they run shorter than the ones still unsolved.

The painted subset is not randomly drawn either: painting is manual, and which changes have been painted is an artifact of the order the annotators worked in. It runs to shorter changes than the dataset as a whole, so the rates of RA2.1 and RA3.2 may move in either direction as painting reaches larger changes. A living dataset also means that the figures of this paper describe one day of it; the scripts that produce them are published with it and re-run on any later state.

The project's popularity says nothing about the quality of its ground truth, and its growth from user reports is so far one content type (Section 3.6).

Finally, the largest fixture holds 198,406 AST nodes, well short of the Robust target's 23,584,040. The target is exercised only by the whole-corpus run of Section 9.4, whose budget and memory cap were applied with eight processes sharing the machine, so a pair near either limit may have crossed it for want of a core rather than of an algorithm. Re-running the timeouts on a quiet machine

DRAFT

moved two of them into the completed set and no more, and the memory kills recur across every commit of the same files, so neither limit looks like an artefact of the sharing.

The judged content changes are fewer and less evenly spread than the code ones. They are the sampled pairs annotation has reached so far, not a draw from them, and the families differ widely in size: 131 pictures but 4 PDFs. We measured no established tool on content (RA5.1), so the content results of Section 9 have no counterpart among the established tools.

11 Related Work

Deltas and their quality. Section 2.1 reviews the work on edit scripts. What it shares is its criterion: a script is better when it is shorter, whether through block moves [59], through more detected moves [18], or through more compact tree scripts [21, 27, 44]. This paper does not propose a better delta. It measures, on changes with a human answer, how far the best delta is from what a reader should be shown (RA2.1–RA2.4), and it finds that the distance is not closed by a better mapping alone.

Visualisations and their quality. The work closest in intent is the work that judges a diff by its reader. srcDiff [16] argues that a delta should follow how developers edit rather than minimise cost and supports this with a user study; ClDiff [31], LSdiff [36] and the refactoring-aware review tools [12, 28, 61] summarise a change at a higher level than its edit script. We add to this line a statement of what a visualisation has to satisfy that is independent of any tool (the invariants), a ground truth of the visualisation itself (the paintings), and the observation that the same distinction governs content that is not code, where perceptual image comparison [64, 67] has made it for pictures already.

Tree-diffing tools. Section 2.4 describes what each of the tools compared here does; what matters for positioning this work is where each one's goal differs from ours. GumTree [23] remains the reference point, and OmniDiff's move-recovery phase adapts its recovery-mappings heuristic directly: the difference is that GumTree targets research use, with broad language and backend flexibility that stays valuable well beyond what RA5.2's single dataset measures, while OmniDiff narrows to a tree-sitter-only frontend and spends the saved generality on production targets. difftastic [32] and diffsitter [20] share that parsing layer but optimise for a diff a human reads comfortably rather than for fidelity of the underlying node mapping, so RA5.2 measures them against an objective they did not choose. Against all of them, Unix `diff` and `git diff` [46] remain the baseline: on our dataset they are the fastest measured, and at the line level they render 59% of the changes perfectly, against 64% for difftastic. BDiff's [43] block awareness does not move that metric, and neither does the choice among `git`'s four algorithms [48].

Ground truth and its ambiguity. GumTree's own evaluation, and most that followed it, score a tool against a single reference mapping per change, and the oracle of Alikhanifard and Tsantalis [3] is the most carefully built of these. RA2.3, RA3.1 and RA3.2 are an argument that this default understates what a ground truth can be asked: on the changes where the correspondence is not one-to-one, where several mappings are equally correct, or where several renderings are, a fixed reference records a correct answer as wrong. The dataset here departs from the default in three ways—a multi-map group, which records an $N{:}M$ site and admits any consistent one-to-one pairing of it; paintings in two presets, with the single painting as the recorded finding that a change has one rendering; and invariants that hold all of it to one convention—and all three were added because annotating real changes forced them, not by design. We are not aware of a code-diffing ground truth that records any of the three.

Empirical studies. Dyer et al. [19] and Lämmel et al. [42] mine AST nodes at scale to study API usage; Arafat and Riehle [4] measure the commit-size distribution of open-source software, the distribution OmniDiff's Fast target is stated against; and Defects4J [35] collects real, reproducible bug fixes from open-source Java projects, each a before/after file pair, and has since served as the file-pair corpus behind most evaluations of AST differencing on Java. This paper's empirical study differs from these in purpose rather than method: it measures what commits change, by content, and how large the changes are, in order to set and then evaluate the targets of a diffing tool.

Diffing content other than code. OmniDiff's container engine takes the model of diffoscope [53], a file as members, each compared by what it is, and applies it to a different end. diffoscope reports every byte that differs, because for a reproducible build [41] every byte matters; OmniDiff decodes only the members whose hash changed and names each change by how much of it a reader would see, so a re-compressed archive or a re-hinted font shows as unchanged. Git's `textconv` [57] reaches the same formats through a converter per format that a user configures by path pattern; OmniDiff recognises the format by its bytes, with no configuration, and diffs a picture as pixels rather than as text. Its picture engine uses pixelmatch's perceptual test [1] per pixel and adds what a diff needs beyond a count of changed pixels: where the changed regions are, and how the frames of an animation align. We are not aware of a ground truth for content diffs that records how much a change changed; the judged changes published with this paper are one.

12 Conclusions

This paper separated the delta of a change, which a machine executes, from its visualisation, which a human reads, and measured the distance between the two on a living ground-truth dataset of changes to code and to six families of other content. The distance is large for every content type we measured. Git shows 9.1% of the files commits change only as differing (RA1.1), and 30.9% of judged changes to such content have a byte delta and nothing deliberate to show (RA2.2). For code, a line script painted as it stands breaks an invariant of visualisation on 88.5% of changes, and a tree script with the human's own mapping on 73.5% (RA2.1); on 13.4% of changes the preferred correspondence cannot be expressed with the four standard operations at all (RA2.3), and on 2.4% the cheapest mapping is not the one humans prefer (RA2.4). Neither the mapping nor its rendering is unique (RA3.1, RA3.2). Established tools meet developers' speed requirements, the line-based ones within tens of milliseconds (RA5.3), and fall short of the human-preferred rendering: no established tool that parses most code changes renders much more than three fifths of them perfectly (RA5.2). OmniDiff diffs 98.8% of changed files as text or by their content, which leaves

PREPRINT

1.2% that a reader sees only as differing bytes, at a speed that meets its target for pictures, catalogs and cursors and not yet for archives, fonts and PDFs.

Based on the results reported in this paper, we recommend the following:

- Evaluate a diffing tool against what a human would mark, not against the length of its edit script: a cheapest script and a preferred account differ measurably.
- Treat the projection from a mapping to what is displayed as a component of its own, with stated rules; a correct mapping does not make a readable diff.
- State the conventions of a hand-made ground truth as machine-checked invariants, and run them on every answer; they find the ground truth's own errors as well as a tool's.
- Record more than one correct answer where there is more than one, in the mapping and in the rendering, and record explicitly where there is exactly one.
- Report accuracy on what reaches the reader: visible nodes and painted bytes for code, and how much of a change is visible for other content.
- Diff a file by what it holds. 9.1% of changed files are binary to git, and for those files a notice that the bytes differ is the whole of today's standard diff.

References

[1] Vladimir Agafonkin. 2015. pixelmatch: A Small, Simple and Fast JavaScript Pixel-Level Image Comparison Library. Mapbox, https://github.com/mapbox/pixelmatch. Accessed 2026-11-09.

[2] Juan Pablo Sandoval Alcocer, Alejandra Cossio-Chavalier, Tiara Rojas-Stambuk, and Leonel Merino. 2023. An Eye-Tracking Study on the Use of Split/Unified Code Change Views for Bug Detection. *IEEE Access* 11 (2023), 136195–136205. https://doi.org/10.1109/ACCESS.2023.3336859

[3] Pouria Alikhanifard and Nikolaos Tsantalis. 2025. A Novel Refactoring and Semantic Aware Abstract Syntax Tree Differencing Tool and a Benchmark for Evaluating the Accuracy of Diff Tools. *ACM Transactions on Software Engineering and Methodology* (2025). https://doi.org/10.1145/3696002

[4] Osama Arafat and Dirk Riehle. 2009. The Commit Size Distribution of Open Source Software. In *2009 42nd Hawaii International Conference on System Sciences*. 1–8. https://doi.org/10.1109/HICSS.2009.421

[5] Muhammad Asaduzzaman, Chanchal K. Roy, Kevin A. Schneider, and Massimiliano Di Penta. 2013. LHDiff: A Language-Independent Hybrid Approach for Tracking Source Code Lines. In *2013 IEEE International Conference on Software Maintenance*. 230–239. https://doi.org/10.1109/ICSM.2013.34

[6] Alberto Bacchelli and Christian Bird. 2013. Expectations, outcomes, and challenges of modern code review. In *Proceedings of the 35th International Conference on Software Engineering (ICSE 2013)*. IEEE, 712–721. https://doi.org/10.1109/ICSE.2013.6606617

[7] Mike Barnett, Christian Bird, João Brunet, and Shuvendu K. Lahiri. 2015. Helping Developers Help Themselves: Automatic Decomposition of Code Review Changesets. In *Proceedings of the 37th IEEE/ACM International Conference on Software Engineering (ICSE 2015)*. IEEE, 134–144. https://doi.org/10.1109/ICSE.2015.35

[8] Tobias Baum, Kurt Schneider, and Alberto Bacchelli. 2019. Associating working memory capacity and code change ordering with code review performance. *Empirical Software Engineering* 24, 4 (2019), 1762–1798. https://doi.org/10.1007/s10664-018-9676-8

[9] Philip Bille. 2005. A Survey on Tree Edit Distance and Related Problems. *Theoretical Computer Science* 337, 1–3 (2005), 217–239. https://doi.org/10.1016/j.tcs.2004.12.030

[10] Hudson Borges and Marco Tulio Valente. 2018. What's in a GitHub Star? Understanding Repository Starring Practices in a Social Coding Platform. *Journal of Systems and Software* 146 (2018), 112–129. https://doi.org/10.1016/j.jss.2018.09.016

[11] Louise Braunstein, Brendan Fallon, Rebecca Lewis, and Kristin Pandit. 2022. Move faster, wait less: Improving code review time at Meta. Meta Engineering blog, https://engineering.fb.com/2022/11/16/culture/meta-code-review-time-improving/. Accessed 2026-09-24.

[12] Rodrigo Brito and Marco Tulio Valente. 2021. RAID: Tool Support for Refactoring-Aware Code Reviews. In *Proceedings of the 29th IEEE/ACM International Conference on Program Comprehension (ICPC 2021)*. IEEE, 265–275. https://doi.org/10.1109/ICPC52881.2021.00033

[13] Max Brunsfeld et al. 2026. Tree-sitter: A Parser Generator Tool and an Incremental Parsing Library. https://tree-sitter.github.io/tree-sitter/. Accessed: 2026-11-10.

[14] Gerardo Canfora, Luigi Cerulo, and Massimiliano Di Penta. 2009. Ldiff: An Enhanced Line Differencing Tool. In *2009 IEEE 31st International Conference on Software Engineering*. 595–598. https://doi.org/10.1109/ICSE.2009.5070564

[15] Sudarshan S. Chawathe, Anand Rajaraman, Hector Garcia-Molina, and Jennifer Widom. 1996. Change Detection in Hierarchically Structured Information. In *Proceedings of the 1996 ACM SIGMOD International Conference on Management of Data*. 493–504. https://doi.org/10.1145/233269.233366

[16] Michael John Decker, Michael L. Collard, L. Gwenn Volkert, and Jonathan I. Maletic. 2020. srcDiff: A syntactic differencing approach to improve the understandability of deltas. *Journal of Software: Evolution and Process* 32, 4, Article e2226 (2020). https://doi.org/10.1002/smr.2226

[17] Marco di Biase, Magiel Bruntink, Arie van Deursen, and Alberto Bacchelli. 2019. The effects of change decomposition on code review—a controlled experiment. *PeerJ Computer Science* 5 (2019), e193. https://doi.org/10.7717/peerj-cs.193

[18] Georg Dotzler and Michael Philippsen. 2016. Move-Optimized Source Code Tree Differencing. In *Proceedings of the 31st IEEE/ACM International Conference on Automated Software Engineering*. 660–671. https://doi.org/10.1145/2970276.2970315

[19] Robert Dyer, Hridesh Rajan, Hoan Anh Nguyen, and Tien N. Nguyen. 2014. Mining billions of AST nodes to study actual and potential usage of Java language features. In *Proceedings of the 36th International Conference on Software Engineering (ICSE 2014)*. Association for Computing Machinery, New York, NY, USA, 779–790. https://doi.org/10.1145/2568225.2568295

[20] Afnan Enayet. 2020. diffsitter: A Tree-Sitter-Based AST Difftool for Meaningful Diffs. https://github.com/afnanenayet/diffsitter. Accessed 2026-07-31.

[21] Jean-Rémy Falleri and Matias Martinez. 2024. Fine-grained, Accurate and Scalable Source Differencing. In *Proceedings of the IEEE/ACM 46th International Conference on Software Engineering*. https://doi.org/10.1145/3597503.3639148

[22] Jean-Rémy Falleri and Matias Martinez. 2024. Replication package of "Fine-grained, accurate and scalable source differencing". Zenodo. https://doi.org/10.5281/zenodo.10474674

[23] Jean-Rémy Falleri, Floréal Morandat, Xavier Blanc, Matias Martinez, and Martin Monperrus. 2014. Fine-grained and accurate source code differencing. In *Proceedings of the 29th ACM/IEEE International Conference on Automated Software Engineering (ASE '14)*. Västerås, Sweden, 313–324. https://doi.org/10.1145/2642937.2642982

[24] Yuanrui Fan, Xin Xia, David Lo, Ahmed E. Hassan, Yuan Wang, and Shanping Li. 2021. A Differential Testing Approach for Evaluating Abstract Syntax Tree Mapping Algorithms. In *2021 IEEE/ACM 43rd International Conference on Software Engineering (ICSE)*. 1174–1185. https://doi.org/10.1109/ICSE43902.2021.00108

[25] Beat Fluri, Michael Würsch, Martin Pinzger, and Harald Gall. 2007. Change Distilling: Tree Differencing for Fine-Grained Source Code Change Extraction. *IEEE Transactions on Software Engineering* 33, 11 (2007), 725–743. https://doi.org/10.1109/TSE.2007.70731

[26] Enrico Fregnan, Larissa Braz, Marco D'Ambros, Gül Çalıklı, and Alberto Bacchelli. 2022. First come first served: the impact of file position on code review. In *Proceedings of the 30th ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering (ESEC/FSE '22)*. Association for Computing Machinery, 483–494. https://doi.org/10.1145/3540250.3549177

[27] Veit Frick, Thomas Grassauer, Fabian Beck, and Martin Pinzger. 2018. Generating Accurate and Compact Edit Scripts Using Tree Differencing. In *2018 IEEE International Conference on Software Maintenance and Evolution (ICSME)*. 264–274. https://doi.org/10.1109/ICSME.2018.00036

[28] Xi Ge, Saurabh Sarkar, Jim Witschey, and Emerson Murphy-Hill. 2017. Refactoring-aware code review. In *2017 IEEE Symposium on Visual Languages and Human-Centric Computing (VL/HCC 2017)*. IEEE, 71–79. https://doi.org/10.1109/VLHCC.2017.8103453

[29] Paul Heckel. 1978. A Technique for Isolating Differences Between Files. *Commun. ACM* 21, 4 (1978), 264–268. https://doi.org/10.1145/359460.359467

[30] Kim Herzig and Andreas Zeller. 2013. The impact of tangled code changes. In *Proceedings of the 10th Working Conference on Mining Software Repositories (MSR 2013)*. IEEE, 121–130. https://doi.org/10.1109/MSR.2013.6624018

[31] Kaifeng Huang, Bihuan Chen, Xin Peng, Daihong Zhou, Ying Wang, Yang Liu, and Wenyun Zhao. 2018. ClDiff: Generating Concise Linked Code Differences. In *Proceedings of the 33rd ACM/IEEE International Conference on Automated Software Engineering*. 679–690. https://doi.org/10.1145/3238147.3238219

[32] Wilfred Hughes. 2018. Difftastic: A Structural Diff That Understands Syntax. https://github.com/Wilfred/difftastic. Accessed 2026-07-31.

[33] James J. Hunt, Kiem-Phong Vo, and Walter F. Tichy. 1998. Delta Algorithms: An Empirical Analysis. *ACM Transactions on Software Engineering and Methodology* 7, 2 (1998), 192–214. https://doi.org/10.1145/279310.279321

Preprint

[34] J. W. Hunt and M. D. McIlroy. 1976. *An Algorithm for Differential File Comparison.* Computing Science Technical Report 41. Bell Laboratories, Murray Hill, New Jersey.

[35] René Just, Darioush Jalali, and Michael D. Ernst. 2014. Defects4J: A Database of Existing Faults to Enable Controlled Testing Studies for Java Programs. In *Proceedings of the 2014 International Symposium on Software Testing and Analysis (ISSTA 2014).* Association for Computing Machinery, New York, NY, USA, 437–440. https://doi.org/10.1145/2610384.2628055

[36] Miryung Kim and David Notkin. 2009. Discovering and Representing Systematic Code Changes. In *2009 IEEE 31st International Conference on Software Engineering.* 309–319. https://doi.org/10.1109/ICSE.2009.5070531

[37] Oleksii Kononenko, Olga Baysal, and Michael W. Godfrey. 2016. Code review quality: how developers see it. In *Proceedings of the 38th International Conference on Software Engineering (ICSE '16).* Association for Computing Machinery, 1028–1038. https://doi.org/10.1145/2884781.2884840

[38] D. Korn, J. MacDonald, J. Mogul, and K. Vo. 2002. *The VCDIFF Generic Differencing and Compression Data Format.* RFC 3284. Internet Engineering Task Force. https://doi.org/10.17487/RFC3284

[39] Yuriy Kotsarenko and Fernando Ramos-Quintana. 2010. Medición de la diferencia de color percibida utilizando el espacio de color de transmisión YIQ NTSC en aplicaciones móviles. *Programación Matemática y Software* 2, 2 (2010), 27–43. https://doi.org/10.30973/progmat/2010.2.2/3

[40] Gunnar Kudrjavets and Ayushi Rastogi. 2023. Does code review speed matter for practitioners? *Empirical Software Engineering* 29, 1, Article 21 (2023). https://doi.org/10.1007/s10664-023-10401-z

[41] Chris Lamb and Stefano Zacchiroli. 2022. Reproducible Builds: Increasing the Integrity of Software Supply Chains. *IEEE Software* 39, 2 (2022), 62–70. https://doi.org/10.1109/MS.2021.3073045

[42] Ralf Lämmel, Ekaterina Pek, and Jürgen Starek. 2011. Large-scale, AST-based API-usage analysis of open-source Java projects. In *Proceedings of the 2011 ACM Symposium on Applied Computing.* 1317–1324.

[43] Yao Lu, Wanwei Liu, Tanghaoran Zhang, Kang Yang, Yang Zhang, Wenyu Xu, Longfei Sun, Xinjun Mao, Shuzheng Gao, and Michael R. Lyu. 2025. BDiff: Block-aware and Accurate Text-based Code Differencing. arXiv preprint arXiv:2510.21094, https://github.com/BDiff/BDiff. Accessed 2026-08-23.

[44] Junnosuke Matsumoto, Yoshiki Higo, and Shinji Kusumoto. 2019. Beyond GumTree: A Hybrid Approach to Generate Edit Scripts. In *2019 IEEE/ACM 16th International Conference on Mining Software Repositories (MSR).* 550–554. https://doi.org/10.1109/MSR.2019.00082

[45] Shane McIntosh, Yasutaka Kamei, Bram Adams, and Ahmed E. Hassan. 2016. An empirical study of the impact of modern code review practices on software quality. *Empirical Software Engineering* 21, 5 (2016), 2146–2189. https://doi.org/10.1007/s10664-015-9381-9

[46] Eugene W. Myers. 1986. An O(ND) Difference Algorithm and Its Variations. *Algorithmica* 1, 2 (1986), 251–266. https://doi.org/10.1007/BF01840446

[47] Jakob Nielsen. 1993. *Usability Engineering.* Morgan Kaufmann, San Francisco, CA.

[48] Yusuf Sulistyo Nugroho, Hideaki Hata, and Kenichi Matsumoto. 2020. How different are different diff algorithms in Git? Use `-histogram` for code changes. *Empirical Software Engineering* 25, 1 (2020), 790–823. https://doi.org/10.1007/s10664-019-09772-z

[49] Mateusz Pawlik and Nikolaus Augsten. 2011. RTED: A Robust Algorithm for the Tree Edit Distance. *Proceedings of the VLDB Endowment* 5, 4 (2011), 334–345. https://doi.org/10.14778/2095686.2095692

[50] Mateusz Pawlik and Nikolaus Augsten. 2015. Efficient Computation of the Tree Edit Distance. *ACM Transactions on Database Systems* 40, 1, Article 3 (2015), 3:1–3:40 pages. https://doi.org/10.1145/2699485

[51] Colin Percival. 2003. Naive Differences of Executable Code. http://www.daemonology.net/bsdiff/. Unpublished.

[52] Md Shamimur Rahman, Zadia Codabux, and Chanchal K. Roy. 2026. Breaking the Alphabet: Rethinking File Ordering in Code Review. In *Proceedings of the 48th IEEE/ACM International Conference on Software Engineering (ICSE '26).* Association for Computing Machinery, 2492–2504. https://doi.org/10.1145/3744916.3787838

[53] Reproducible Builds. [n. d.]. diffoscope: In-Depth Comparison of Files, Archives, and Directories. https://diffoscope.org. Accessed 2026-11-09.

[54] Caitlin Sadowski, Emma Söderberg, Luke Church, Michal Sipko, and Alberto Bacchelli. 2018. Modern code review: a case study at Google. In *Proceedings of the 40th International Conference on Software Engineering: Software Engineering in Practice (ICSE-SEIP '18).* Association for Computing Machinery, 181–190. https://doi.org/10.1145/3183519.3183525

[55] Kuo-Chung Tai. 1979. The Tree-to-Tree Correction Problem. *J. ACM* 26, 3 (1979), 422–433. https://doi.org/10.1145/322139.322143

[56] Yida Tao, Yingnong Dang, Tao Xie, Dongmei Zhang, and Sunghun Kim. 2012. How do software engineers understand code changes? An exploratory study in industry. In *Proceedings of the ACM SIGSOFT 20th International Symposium on the Foundations of Software Engineering (FSE '12).* Association for Computing Machinery, Article 51, 11 pages. https://doi.org/10.1145/2393596.2393656

[57] The Git Project. [n. d.]. gitattributes: Defining Attributes per Path. Git documentation, https://git-scm.com/docs/gitattributes. Sections "Marking files as binary" and "Performing text diffs of binary files". Accessed 2026-11-09.

[58] The Git Project. 2026. gitformat-pack: Git Pack Format. https://git-scm.com/docs/gitformat-pack. Accessed: 2026-11-10.

[59] Walter F. Tichy. 1984. The String-to-String Correction Problem with Block Moves. *ACM Transactions on Computer Systems* 2, 4 (1984), 309–321. https://doi.org/10.1145/357401.357404

[60] Andrew Tridgell and Paul Mackerras. 1996. *The rsync Algorithm.* Technical Report TR-CS-96-05. Department of Computer Science, The Australian National University.

[61] Nikolaos Tsantalis, Ameya Ketkar, and Danny Dig. 2022. RefactoringMiner 2.0. *IEEE Transactions on Software Engineering* 48, 3 (2022), 930–950. https://doi.org/10.1109/TSE.2020.3007722

[62] V. Vyšniauskas. 2009. Anti-aliased Pixel and Intensity Slope Detector. *Elektronika ir Elektrotechnika* 95, 7 (2009), 107–110.

[63] Robert A. Wagner and Michael J. Fischer. 1974. The String-to-String Correction Problem. *J. ACM* 21, 1 (1974), 168–173. https://doi.org/10.1145/321796.321811

[64] Zhou Wang, Alan C. Bovik, Hamid R. Sheikh, and Eero P. Simoncelli. 2004. Image Quality Assessment: From Error Visibility to Structural Similarity. *IEEE Transactions on Image Processing* 13, 4 (2004), 600–612. https://doi.org/10.1109/TIP.2003.819861

[65] Pavlína Wurzel Gonçalves, Enrico Fregnan, Tobias Baum, Kurt Schneider, and Alberto Bacchelli. 2022. Do explicit review strategies improve code review performance? Towards understanding the role of cognitive load. *Empirical Software Engineering* 27, 4, Article 92 (2022). https://doi.org/10.1007/s10664-022-10123-8

[66] Wuu Yang. 1991. Identifying Syntactic Differences Between Two Programs. *Software: Practice and Experience* 21, 7 (1991), 739–755. https://doi.org/10.1002/spe.4380210706

[67] Hector Yee. 2004. Perceptual Metric for Production Testing. *Journal of Graphics Tools* 9, 4 (2004), 33–40. https://doi.org/10.1080/10867651.2004.10504900

[68] Kaizhong Zhang and Dennis Shasha. 1989. Simple Fast Algorithms for the Editing Distance between Trees and Related Problems. *SIAM J. Comput.* 18, 6 (1989), 1245–1262. https://doi.org/10.1137/0218082

PREPRINT